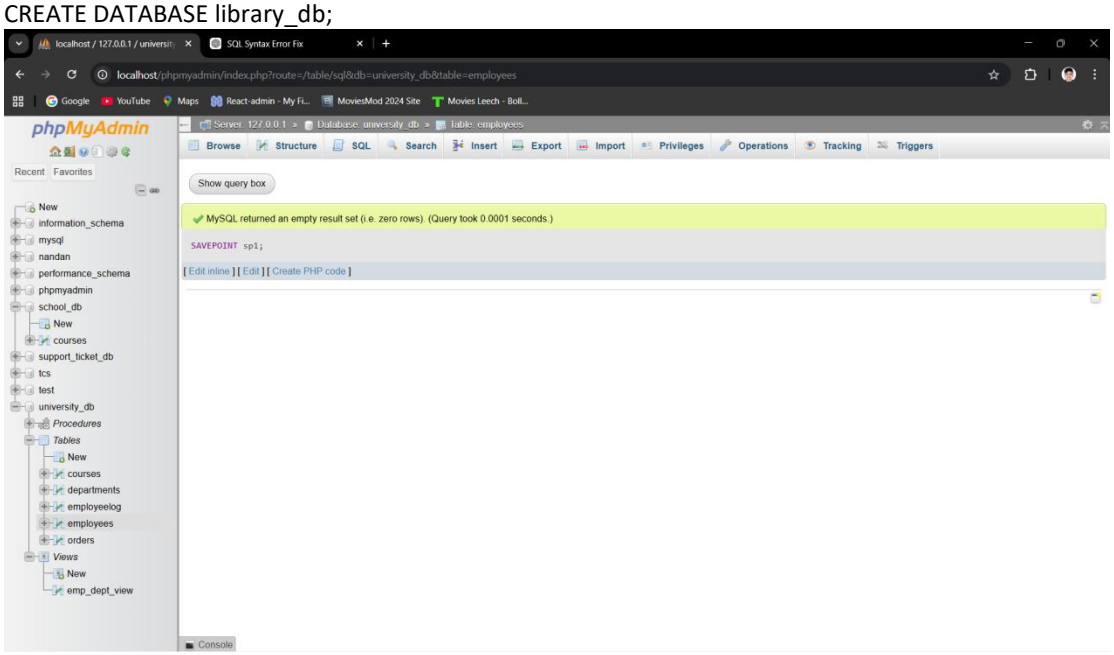
1. Introduction to SQL

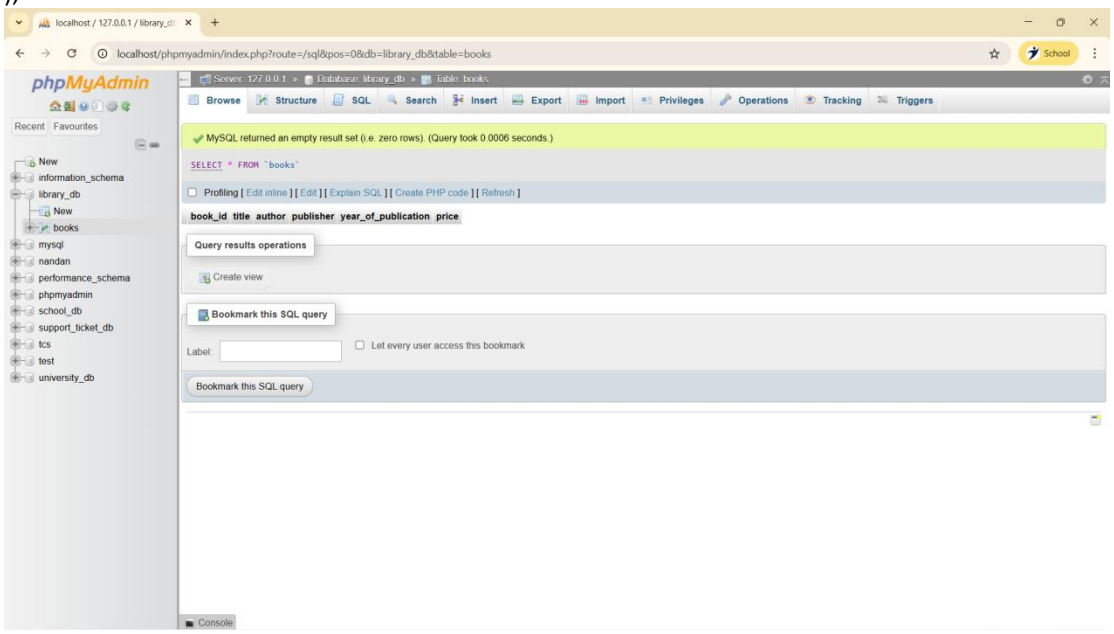
Lab 3: Create a database called library_db and a table books with columns: book_id, title, author, publisher, year_of_publication, and price. Insert five records.

CREATE DATABASE library_db;



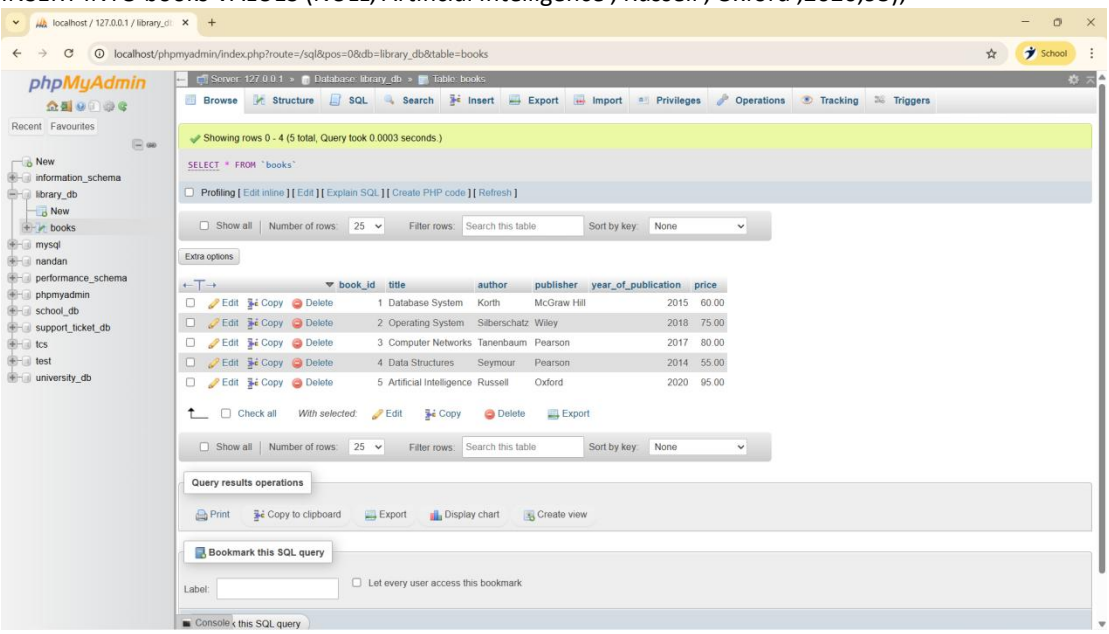
USE library_db;

```
CREATE TABLE books (
   book_id INT PRIMARY KEY AUTO_INCREMENT,
   title VARCHAR(100),
   author VARCHAR(50),
   publisher VARCHAR(50),
   year_of_publication INT,
   price DECIMAL(10,2)
);
```



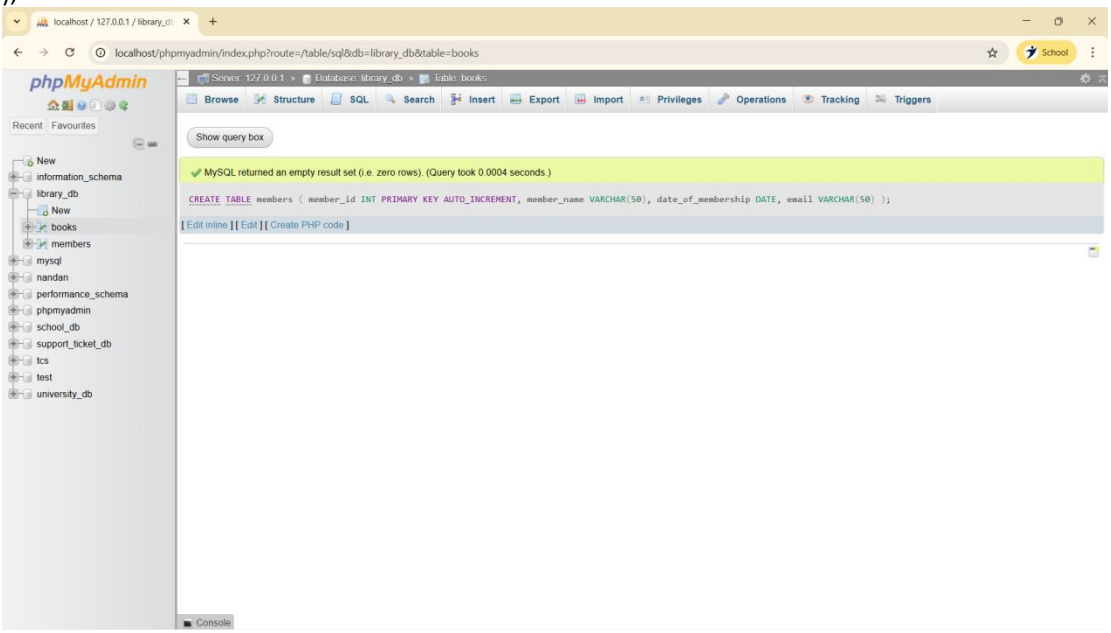
INSERT INTO books VALUES (NULL,'Database System','Korth','McGraw Hill',2015,60);
INSERT INTO books VALUES (NULL,'Operating System','Silberschatz','Wiley',2018,75);
INSERT INTO books VALUES (NULL,'Computer Networks','Tanenbaum','Pearson',2017,80);

INSERT INTO books VALUES (NULL,'Data Structures','Seymour','Pearson',2014,55);
INSERT INTO books VALUES (NULL,'Artificial Intelligence','Russell','Oxford',2020,95);

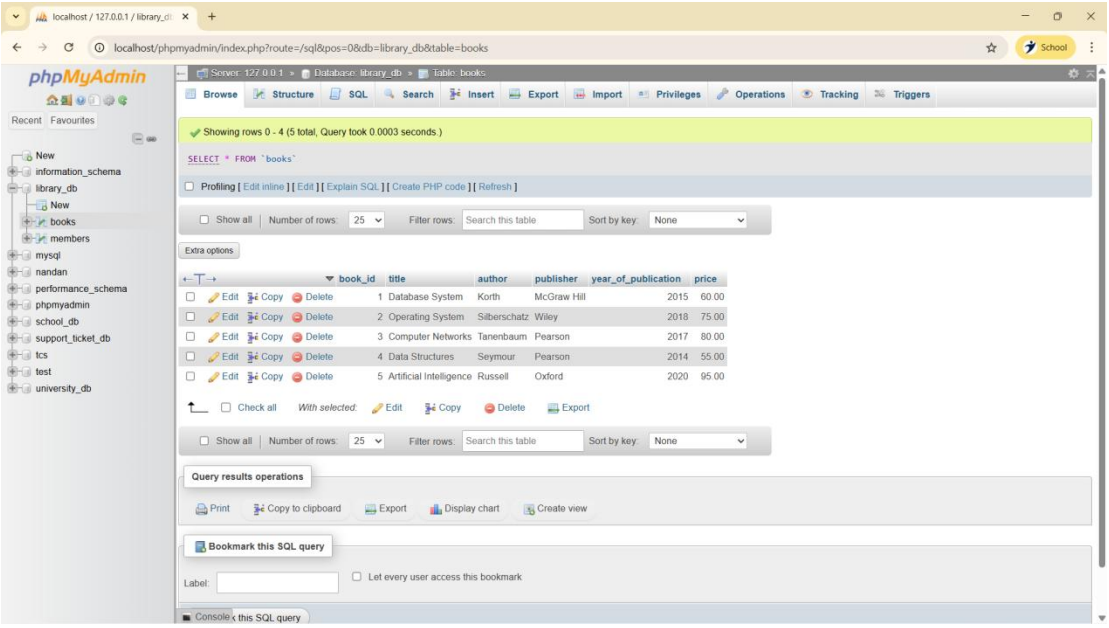


Lab 4: Create a table members in library_db and insert five records.

```
CREATE TABLE members (
   member_id INT PRIMARY KEY AUTO_INCREMENT,
   member_name VARCHAR(50),
   date_of_membership DATE,
   email VARCHAR(50)
);
```



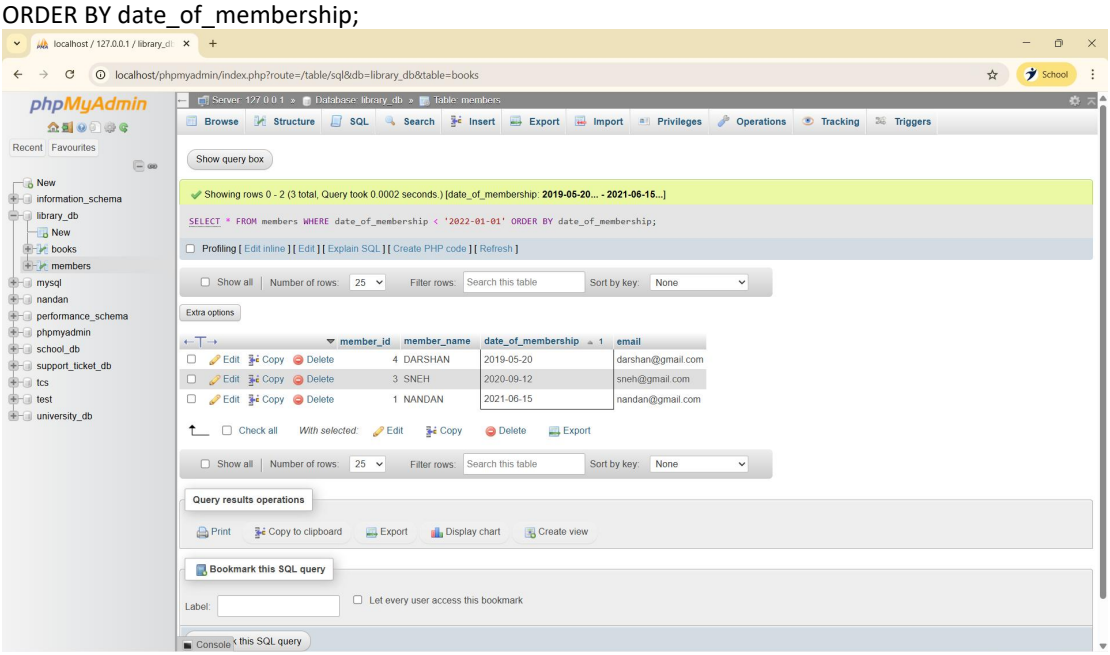
INSERT INTO members VALUES (NULL,'NANDAN','2021-06-15','nandan@gmail.com');
INSERT INTO members VALUES (NULL,'YASH','2023-02-10','yash@gmail.com');
INSERT INTO members VALUES (NULL,'SNEH','2020-09-12','sneh@gmail.com');
INSERT INTO members VALUES (NULL,'DARSHAN','2019-05-20','darshan@gmail.com');
INSERT INTO members VALUES (NULL,'JEEL','2022-01-25','jeel@gmail.com');



2. SQL Syntax

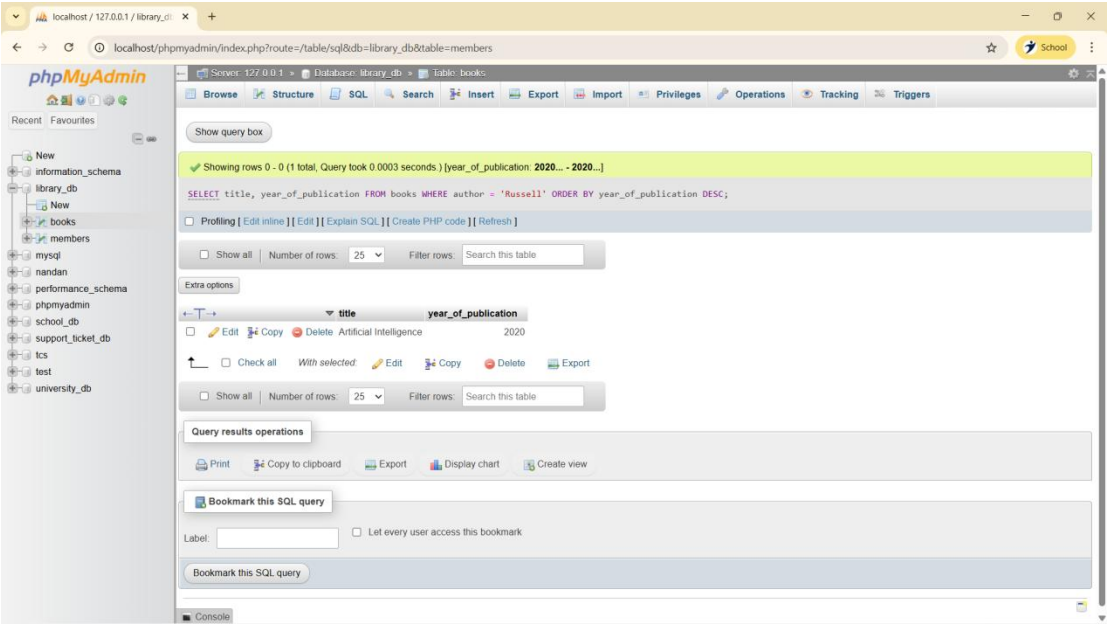
Lab 3: Retrieve members who joined before 2022.

```
SELECT * FROM members
WHERE date_of_membership < '2022-01-01'
ORDER BY date_of_membership;
```



Lab 4: Display titles of books by specific author sorted by year descending.

```
SELECT title, year_of_publication
FROM books
WHERE author = 'Russell'
ORDER BY year_of_publication DESC;
```



3. SQL Constraints

Lab 3: Add CHECK constraint to ensure price > 0.

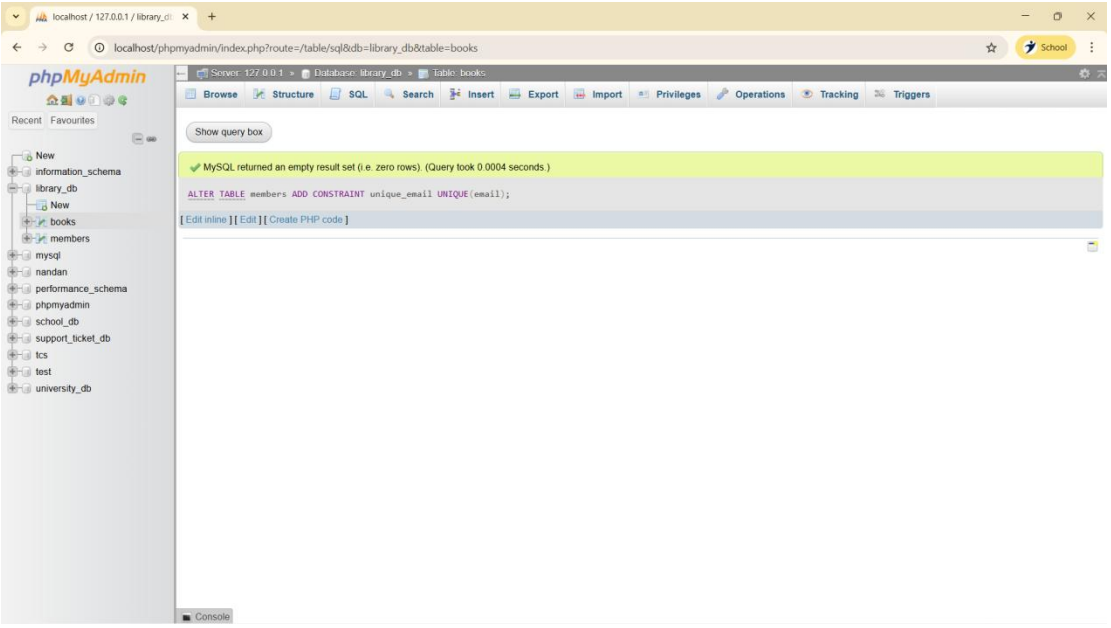
ALTER TABLE books

ADD CONSTRAINT chk_price CHECK (price > 0);

Lab 4: Add UNIQUE constraint on email in members table.

ALTER TABLE members

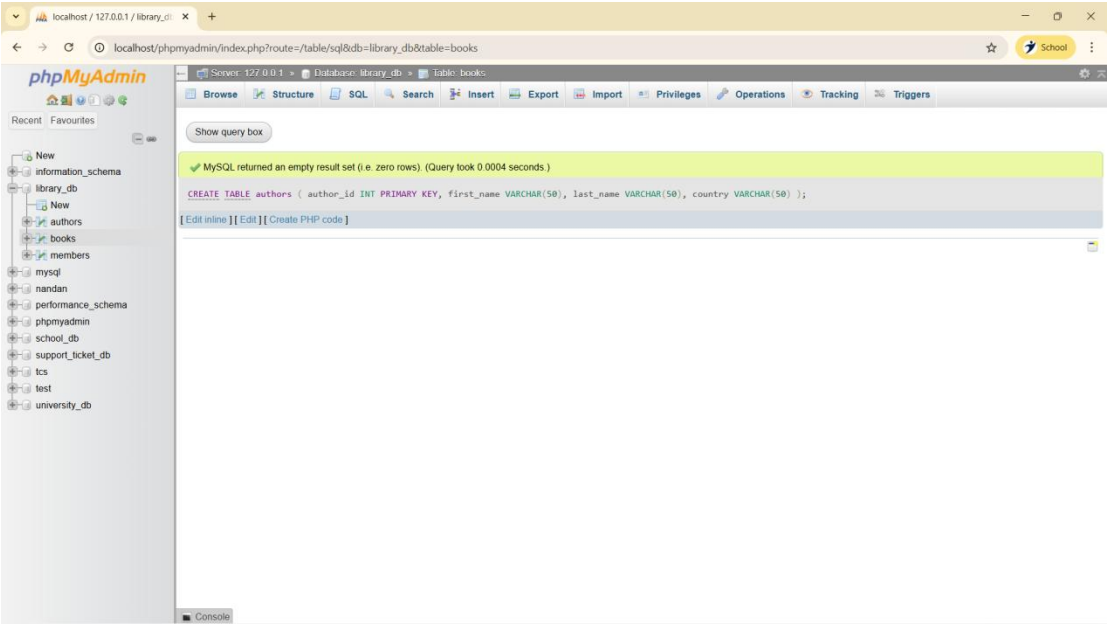
ADD CONSTRAINT unique_email UNIQUE(email);



4. Main SQL Commands and Sub-commands (DDL)

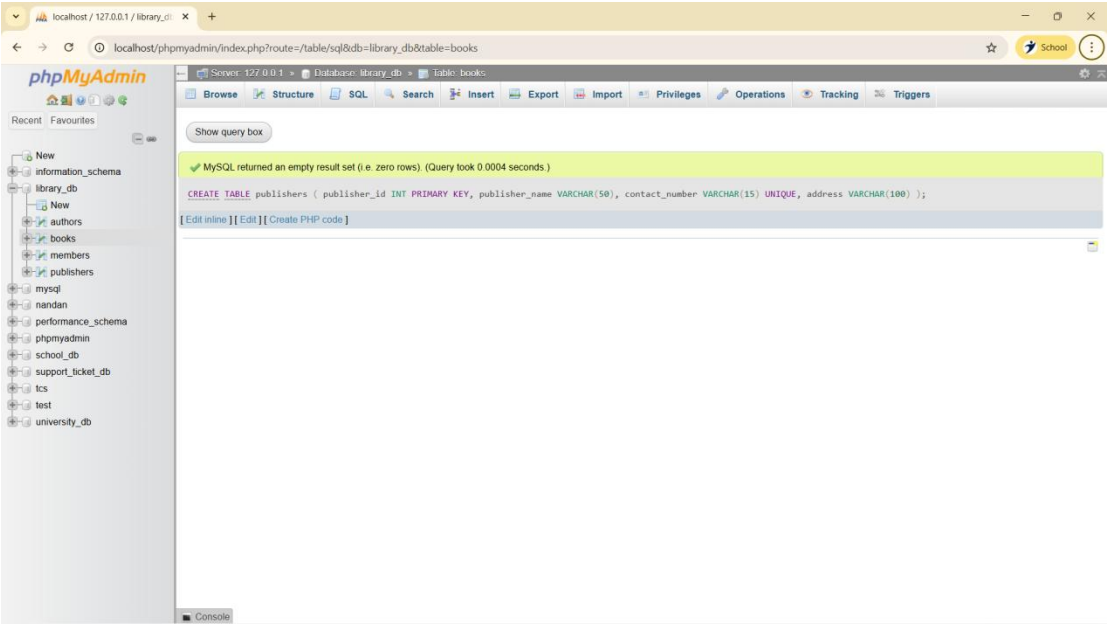
Lab 3: Create authors table.

```
CREATE TABLE authors (
   author_id INT PRIMARY KEY,
   first_name VARCHAR(50),
   last_name VARCHAR(50),
   country VARCHAR(50)
);
```



Lab 4: Create publishers table.

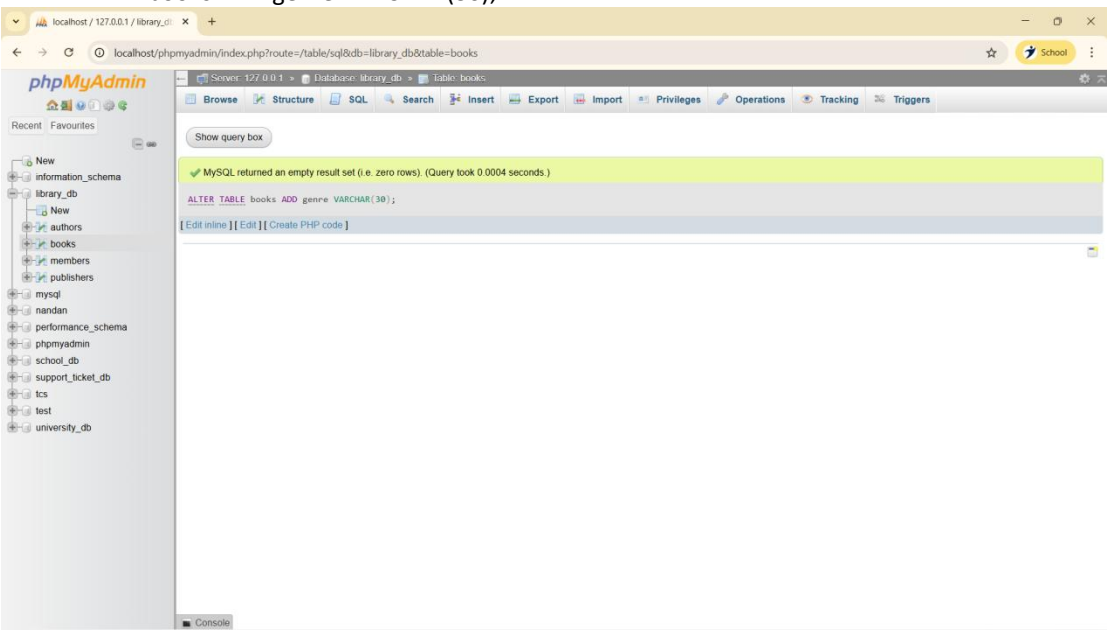
```
CREATE TABLE publishers (
   publisher_id INT PRIMARY KEY,
   publisher_name VARCHAR(50),
   contact_number VARCHAR(15) UNIQUE,
   address VARCHAR(100)
);
```



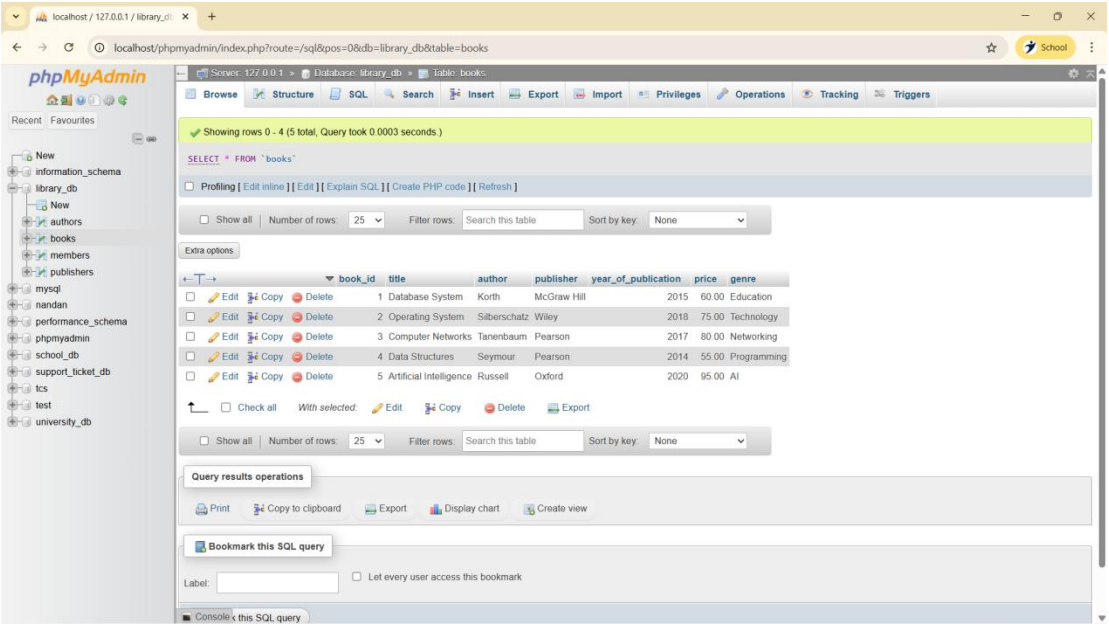
5. ALTER Command

Lab 3: Add genre column and update records.

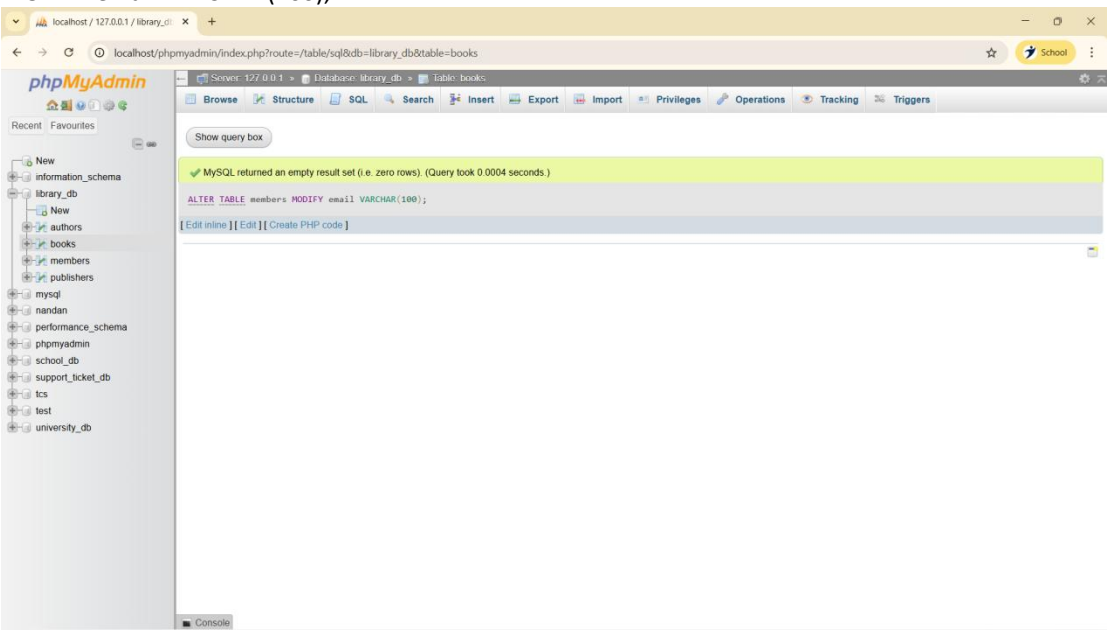
ALTER TABLE books ADD genre VARCHAR(30);



UPDATE books SET genre='Education' WHERE book_id=1;
UPDATE books SET genre='Technology' WHERE book_id=2;
UPDATE books SET genre='Networking' WHERE book_id=3;
UPDATE books SET genre='Programming' WHERE book_id=4;
UPDATE books SET genre='AI' WHERE book_id=5;

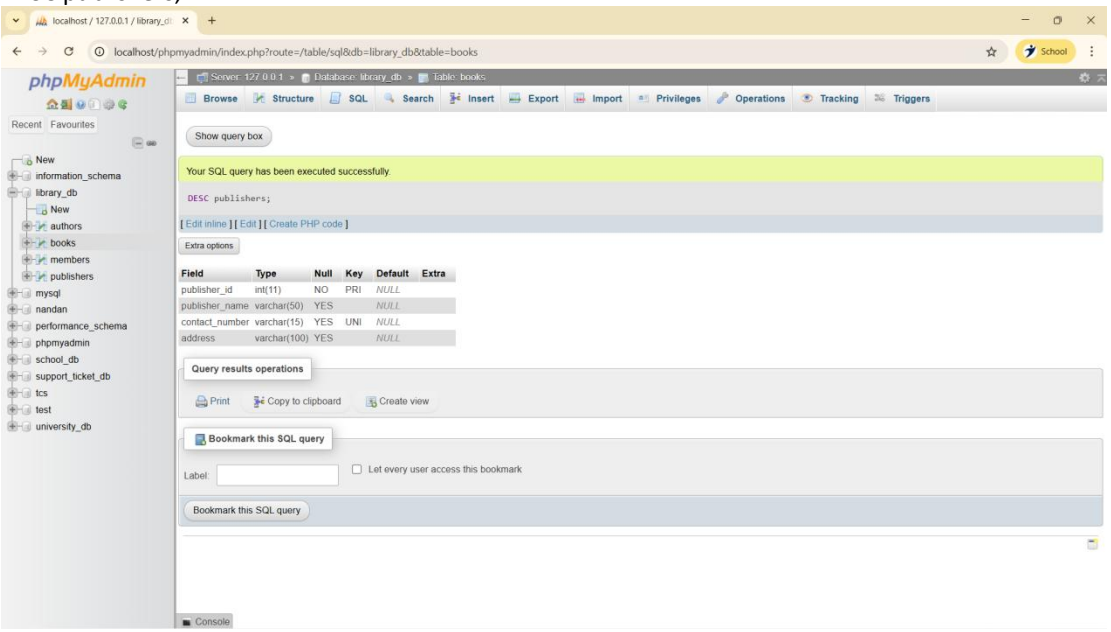


Lab 4: Increase email column length.
ALTER TABLE members
MODIFY email VARCHAR(100);

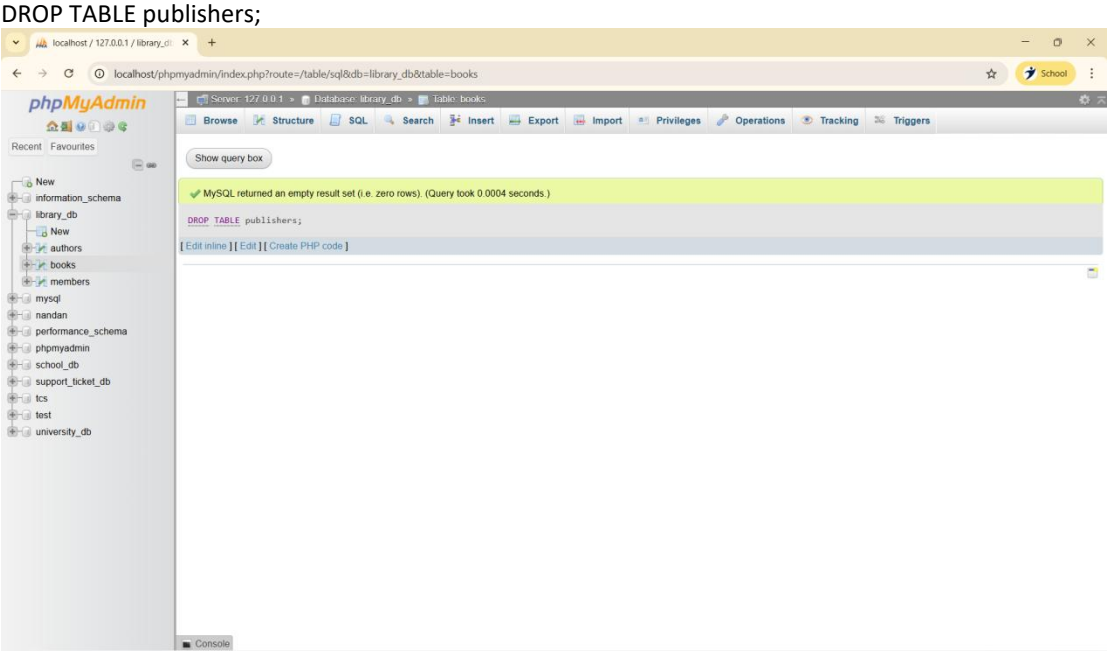


6. DROP Command
Lab 3: Drop publishers table.

DESC publishers;

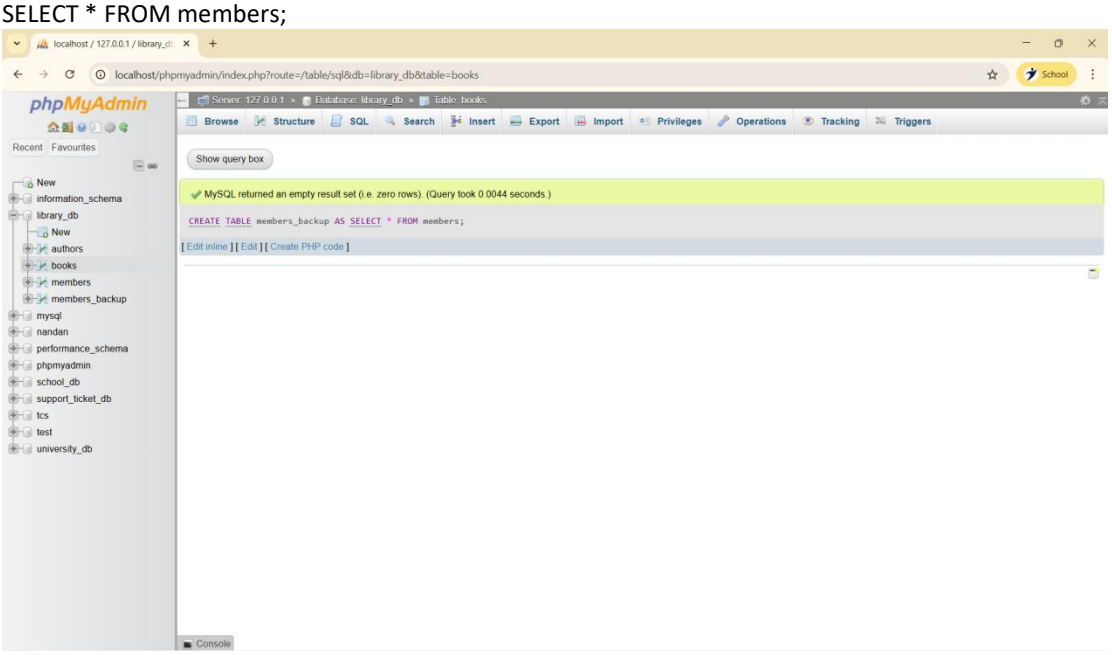


DROP TABLE publishers;

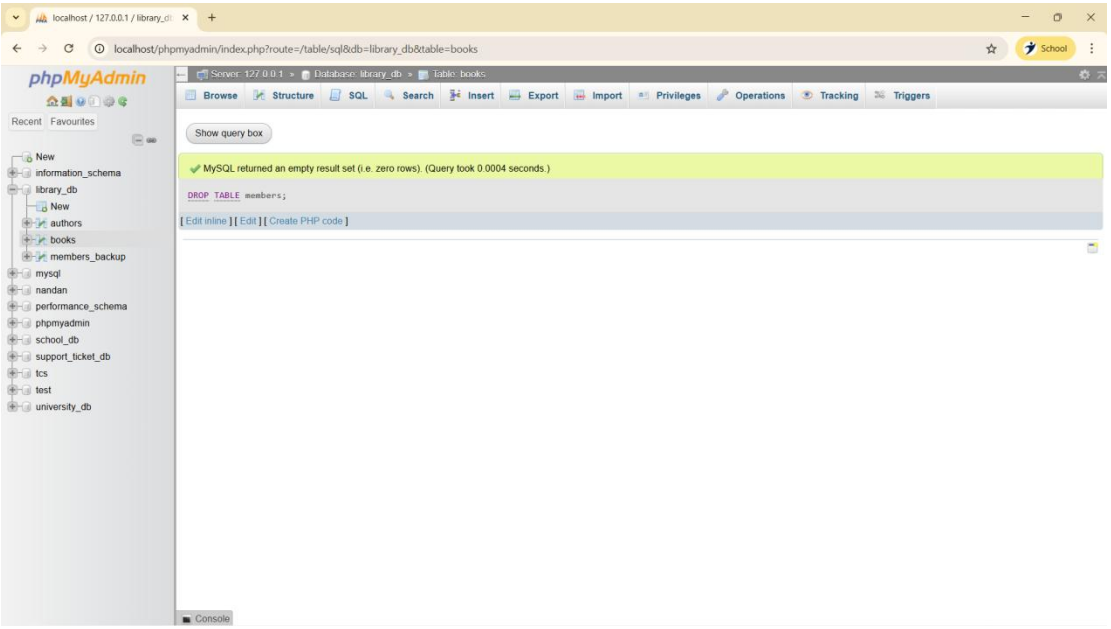


Lab 4: Backup members table then drop original.

CREATE TABLE members_backup AS
SELECT * FROM members;

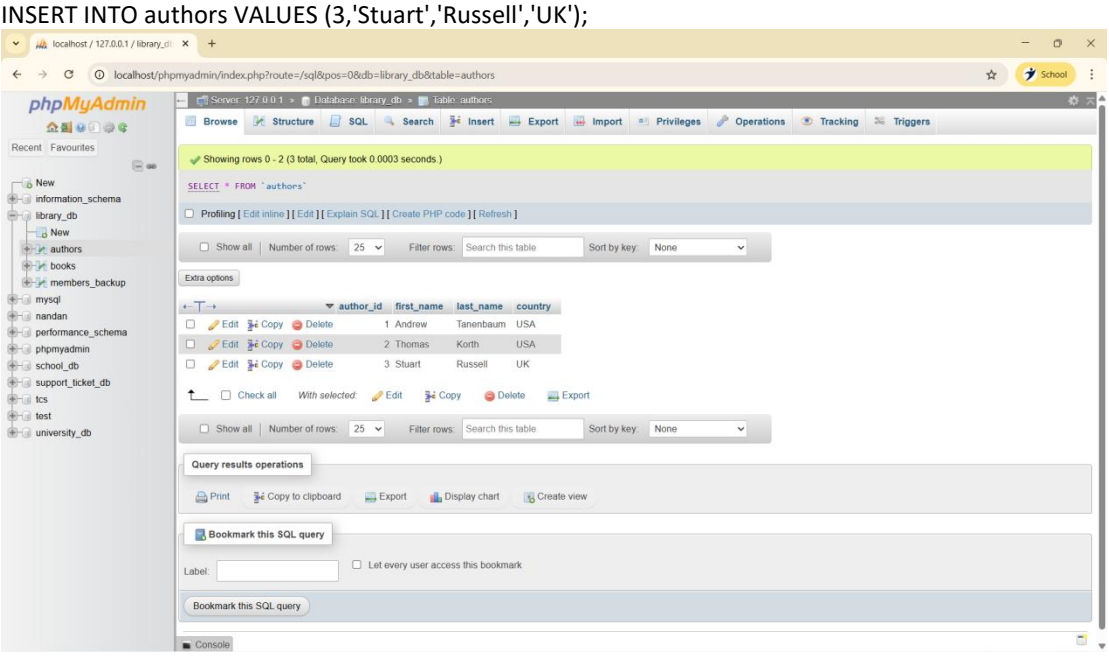


DROP TABLE members;

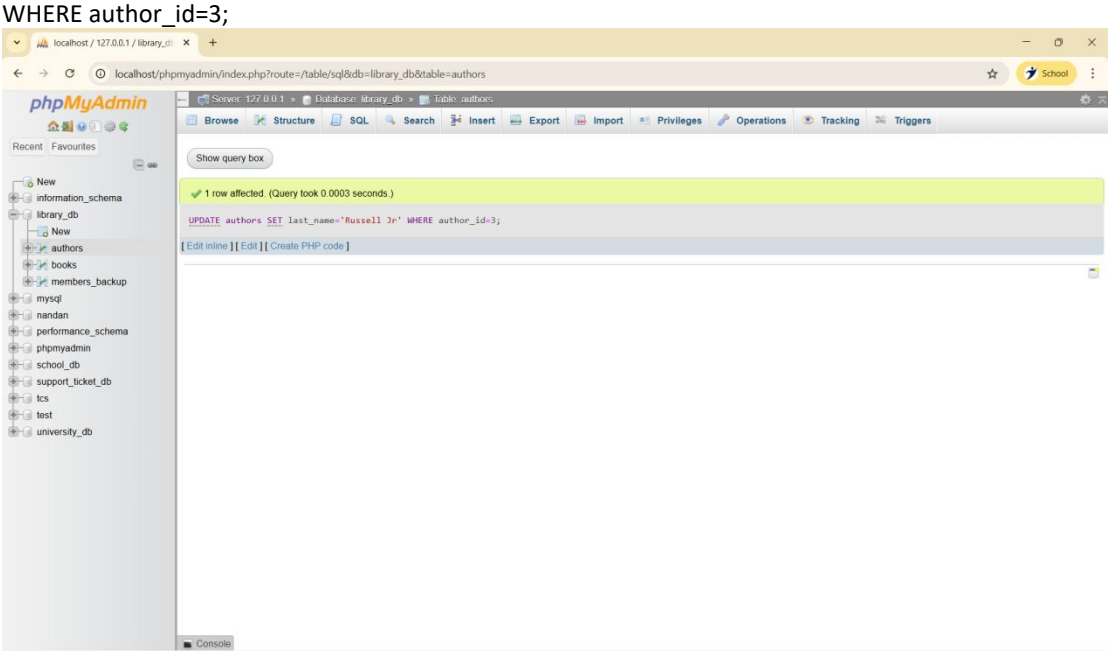


7. Data Manipulation Language (DML)
Lab 4: Insert authors and update last name.
INSERT INTO authors VALUES (1,'Andrew','Tanenbaum','USA');
INSERT INTO authors VALUES (2,'Thomas','Korth','USA');

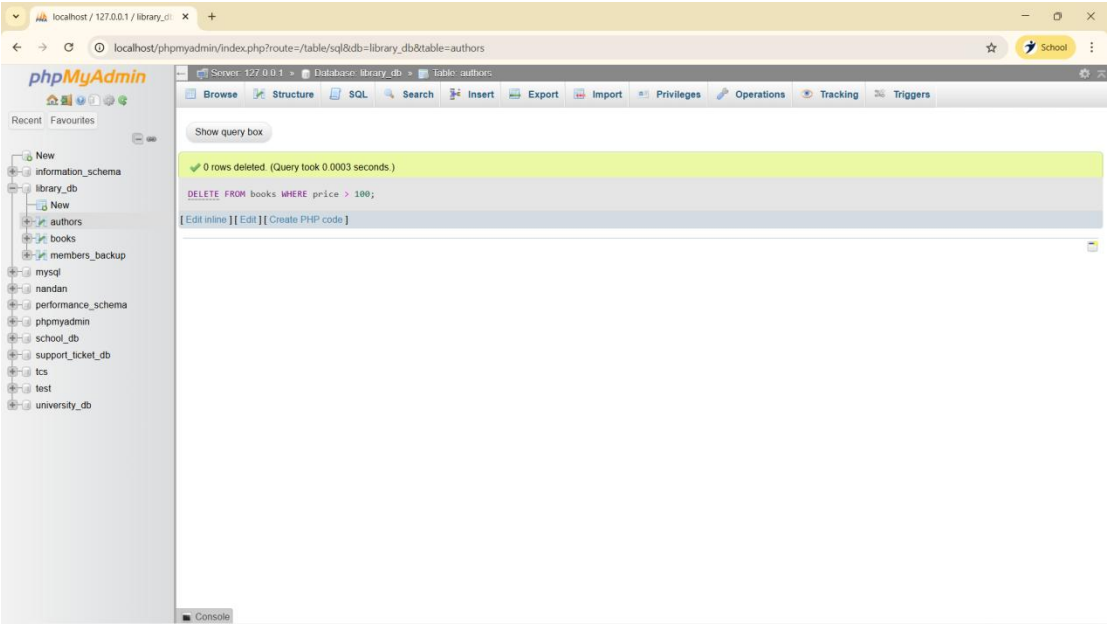
INSERT INTO authors VALUES (3,'Stuart','Russell','UK');



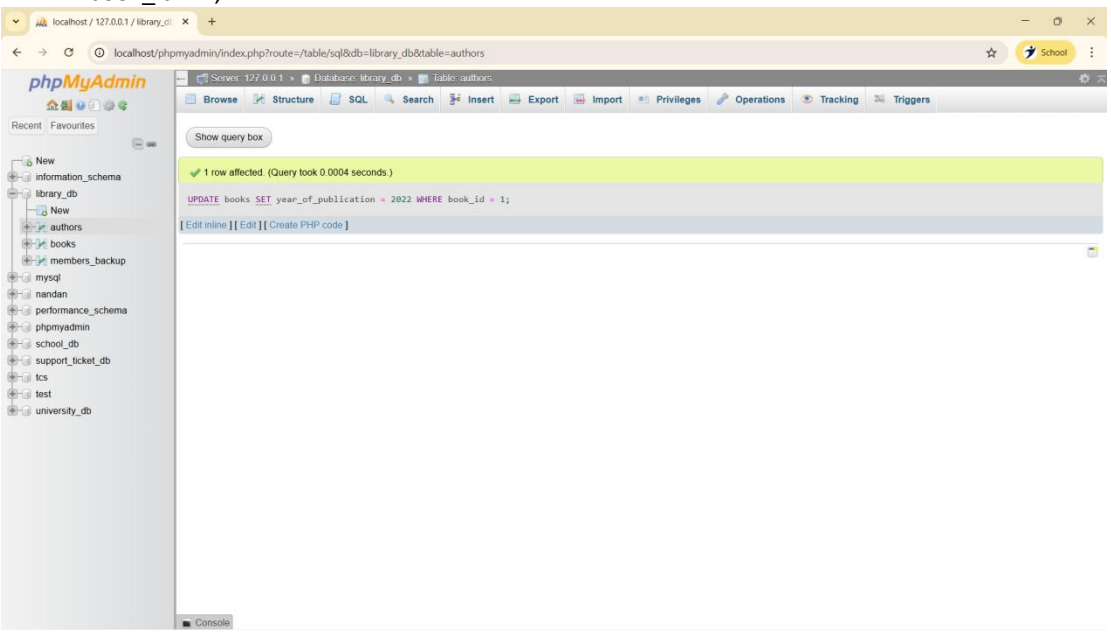
UPDATE authors
SET last_name='Russell Jr'
WHERE author_id=3;



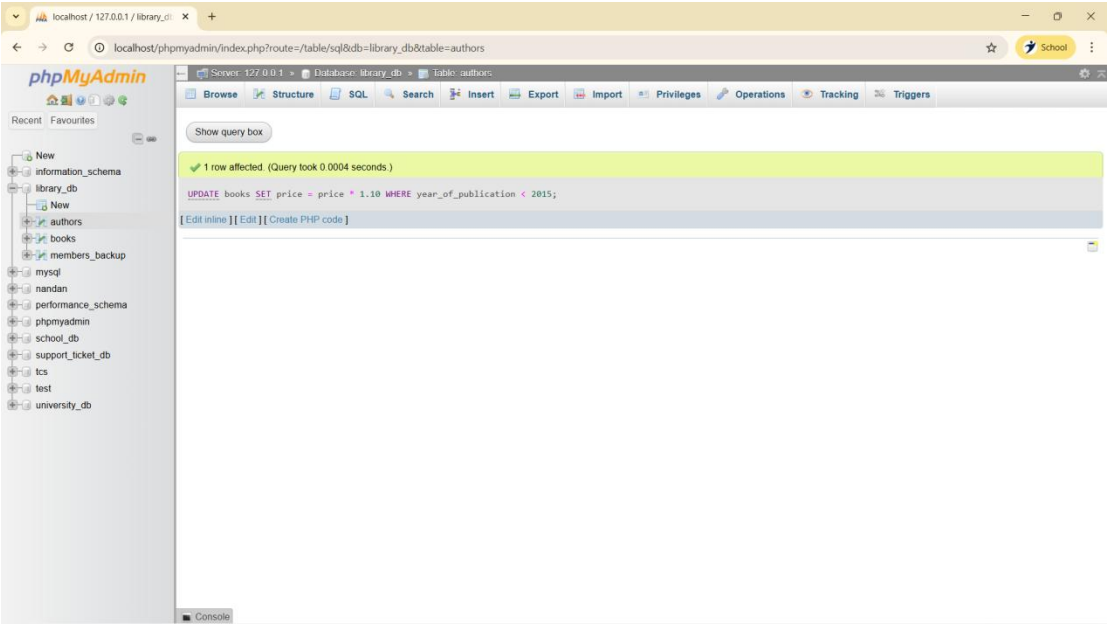
Lab 5: Delete book where price > 100
DELETE FROM books
WHERE price > 100;



8. UPDATE Command
Lab 3: Update publication year.
UPDATE books
SET year_of_publication = 2022
WHERE book_id = 1;

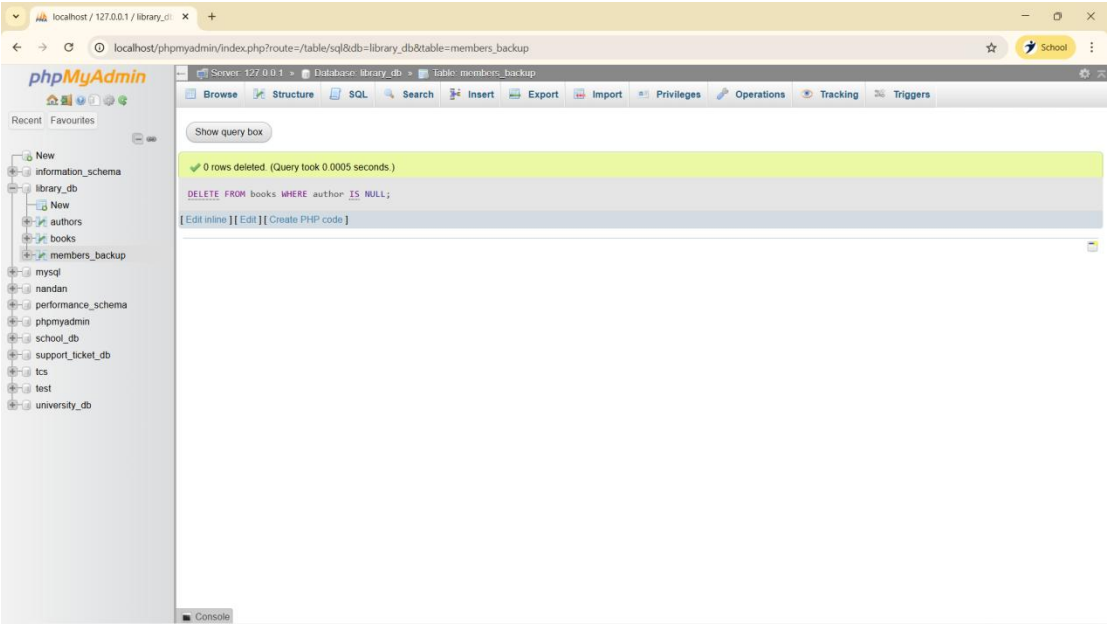


Lab 4: Increase price of books before 2015 by 10%.
UPDATE books
SET price = price * 1.10
WHERE year_of_publication < 2015;

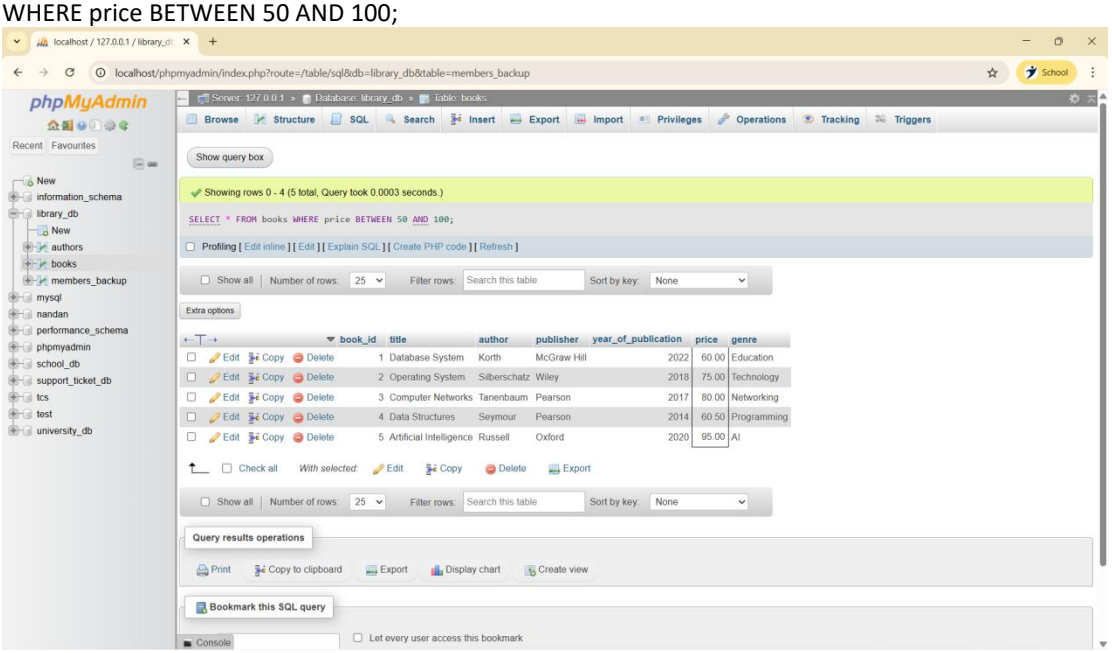


9. DELETE Command
Lab 3: Remove members who joined before 2020.
DELETE FROM members
WHERE date_of_membership < '2020-01-01';

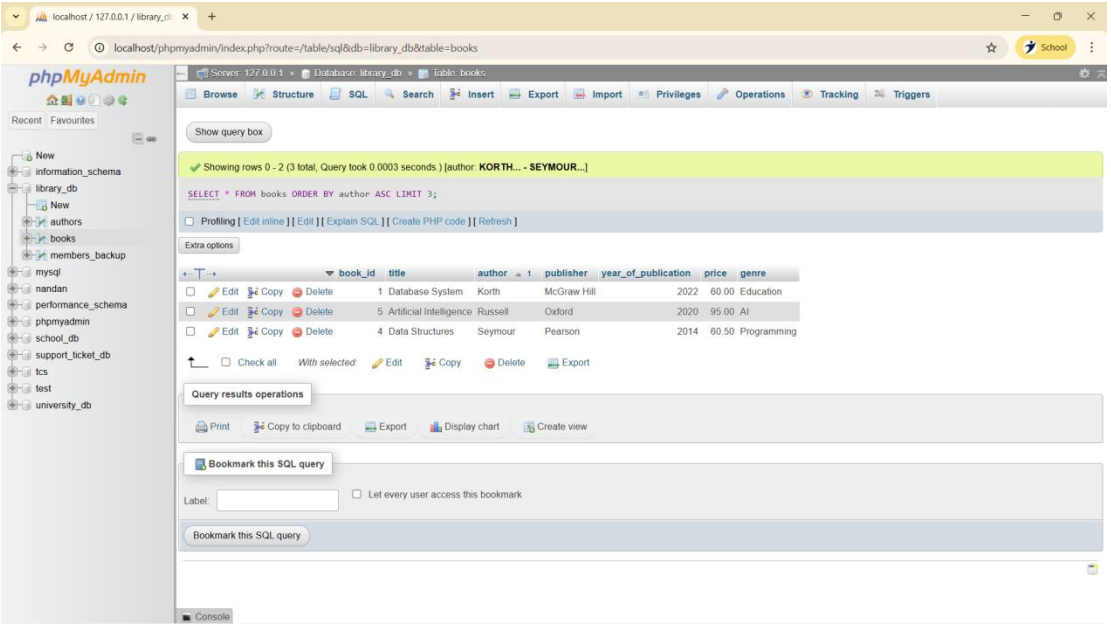
Lab 4: Delete books with NULL author.
DELETE FROM books
WHERE author IS NULL;



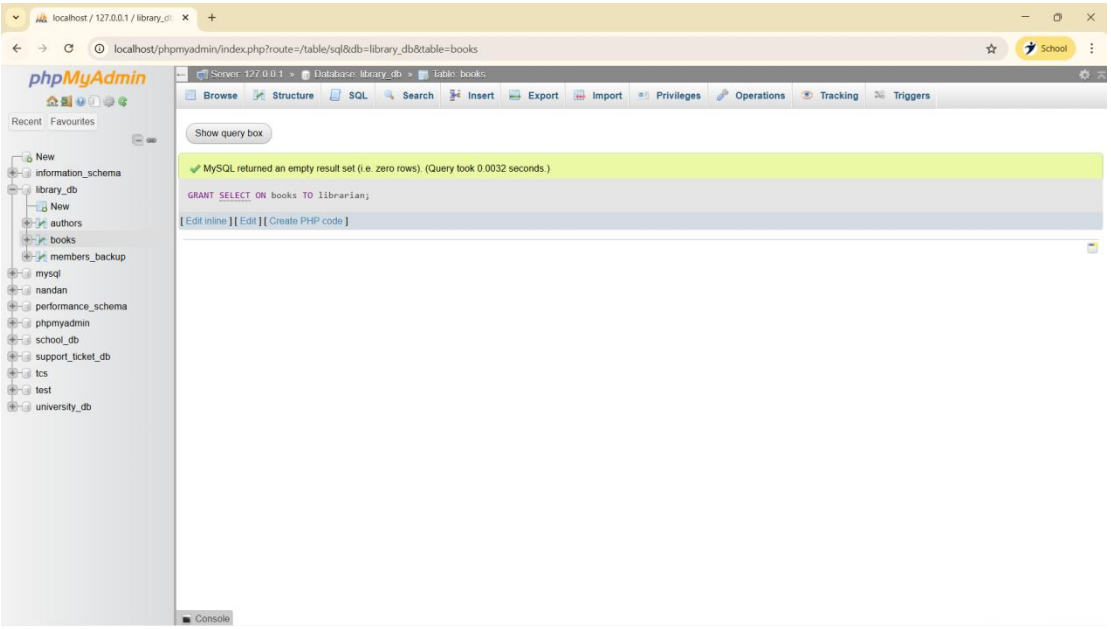
10. Data Query Language (DQL)
Lab 4: Retrieve books with price between 50 and 100.
SELECT * FROM books
WHERE price BETWEEN 50 AND 100;



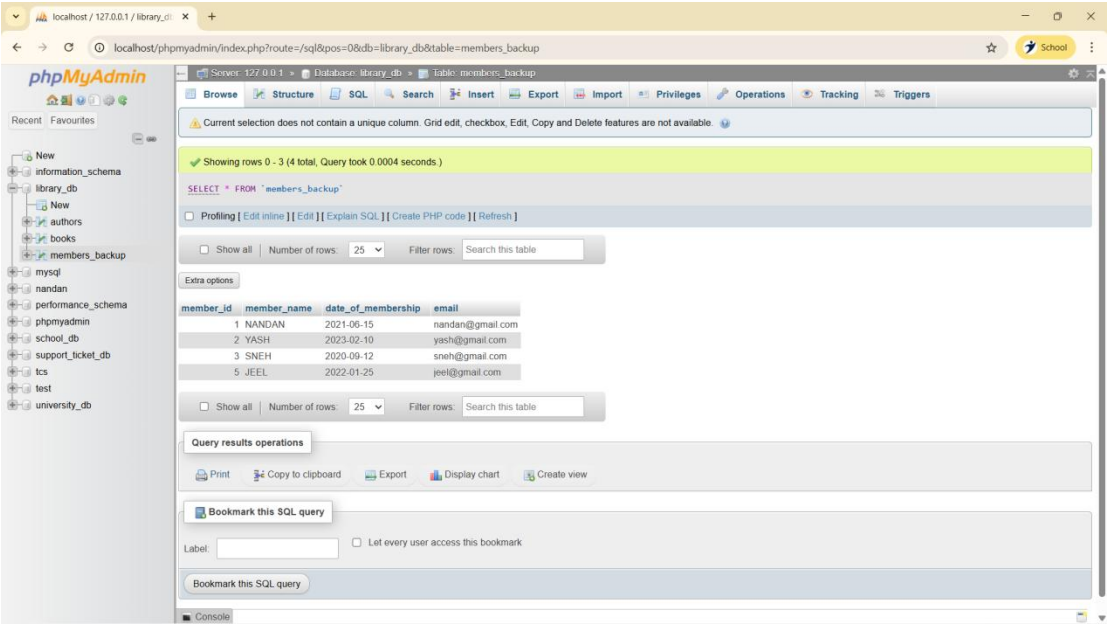
Lab 5: Retrieve books sorted by author (limit 3).
SELECT * FROM books
ORDER BY author ASC
LIMIT 3;



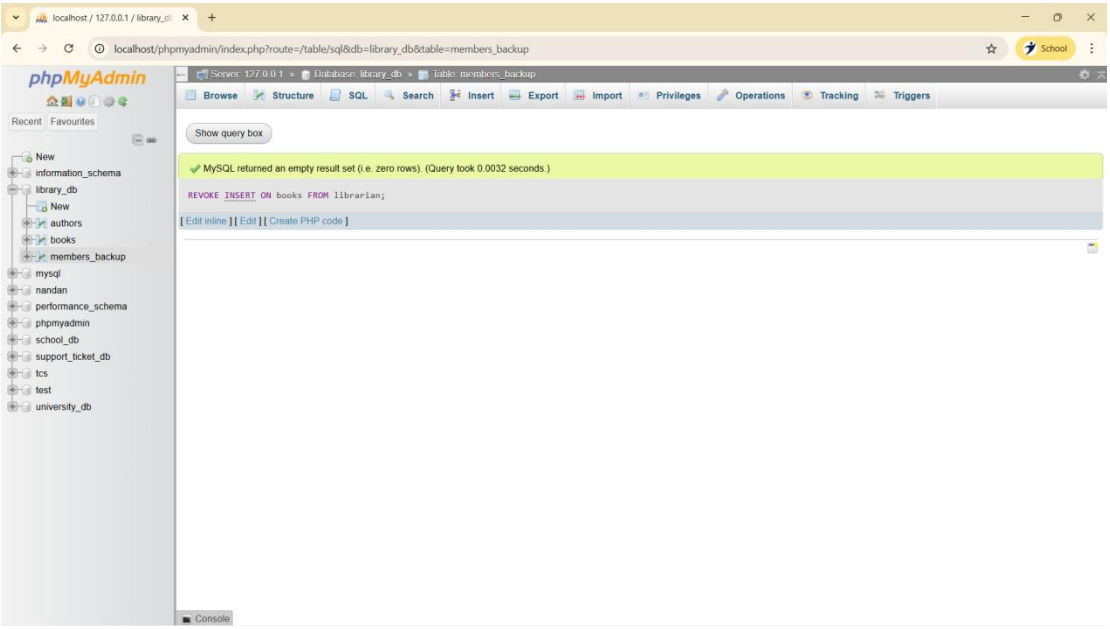
11. Data Control Language (DCL)
Lab 3: Grant SELECT permission.
GRANT SELECT
ON books
TO librarian;



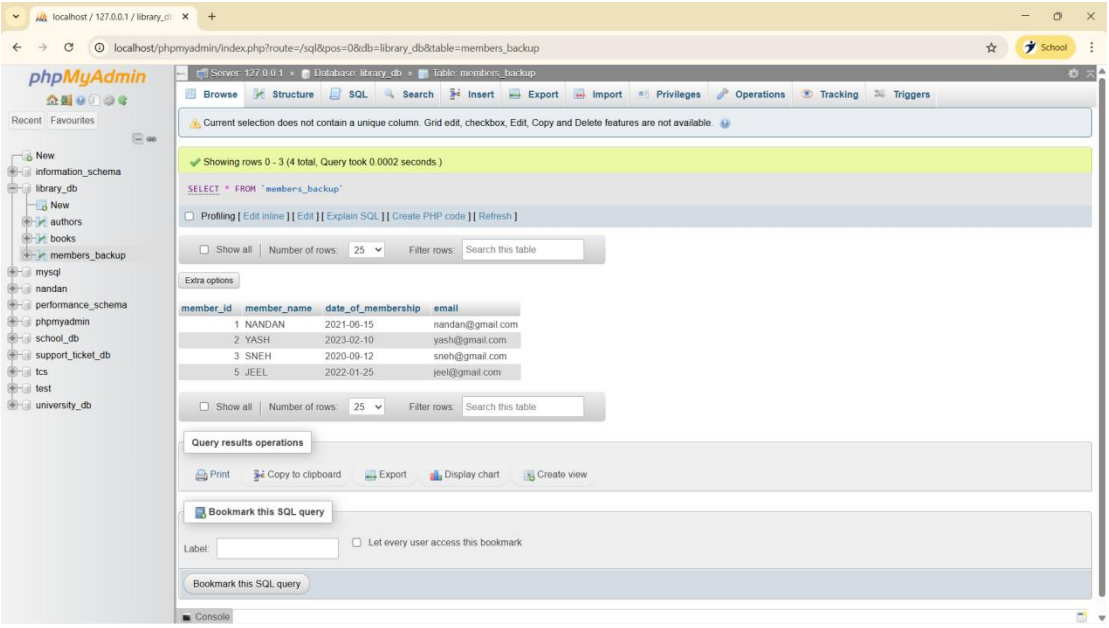
Lab 4: Grant INSERT and UPDATE permissions.
GRANT INSERT, UPDATE
ON members
TO admin;



12. REVOKE Command
Lab 3: Revoke INSERT privilege.
REVOKE INSERT
ON books
FROM librarian;



Lab 4: Revoke all permissions.
REVOKE ALL PRIVILEGES
ON members
FROM admin;

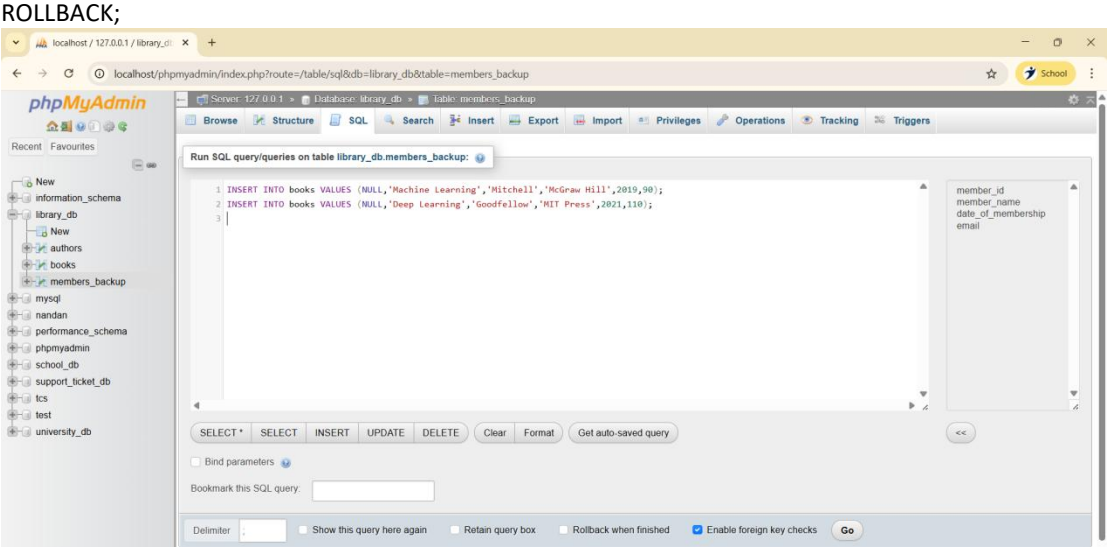


13. Transaction Control Language (TCL)
Lab 3: COMMIT and ROLLBACK example.
INSERT INTO books VALUES (NULL,'Machine Learning','Mitchell','McGraw Hill',2019,90);
INSERT INTO books VALUES (NULL,'Deep Learning','Goodfellow','MIT Press',2021,110);

COMMIT;

INSERT INTO books VALUES (NULL,'Test Book','ABC','XYZ',2024,50);

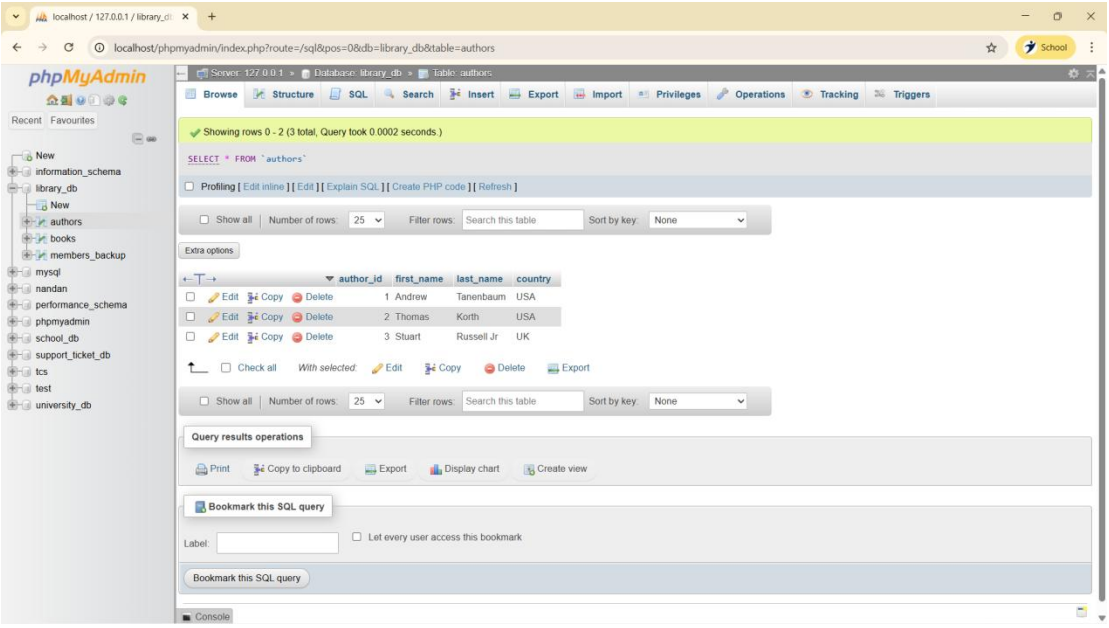
ROLLBACK;



Lab 4: SAVEPOINT example.
SAVEPOINT before_update;

UPDATE members SET member_name='Updated Name' WHERE member_id=1;

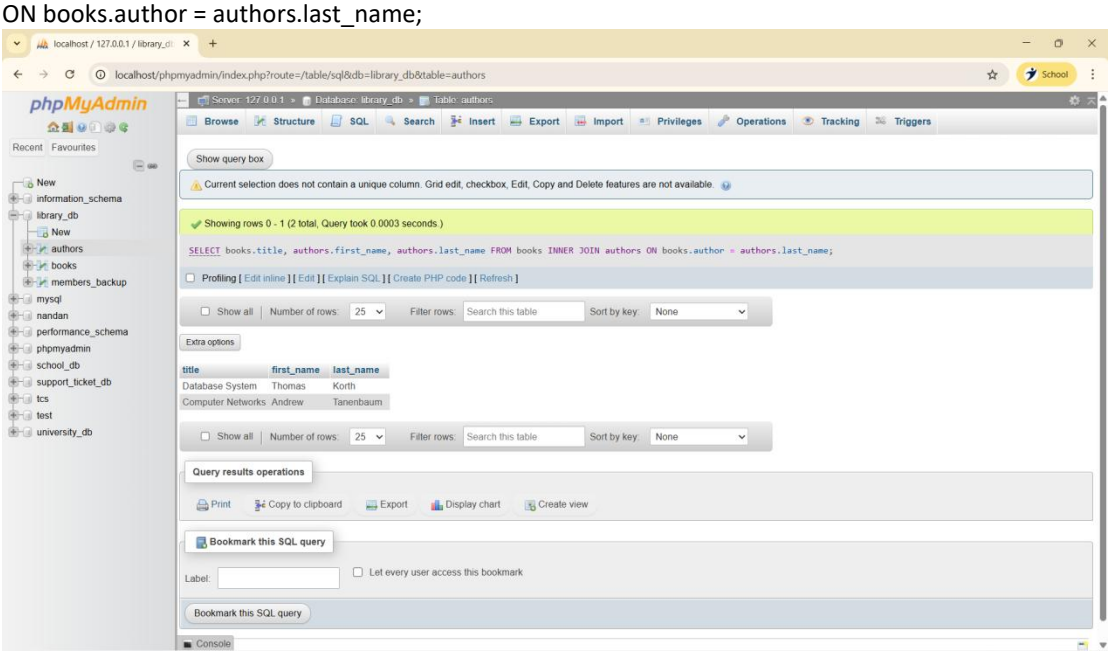
ROLLBACK TO before_update;



14. SQL Joins

Lab 3: INNER JOIN books and authors.

SELECT books.title, authors.first_name, authors.last_name
FROM books
INNER JOIN authors
ON books.author = authors.last_name;



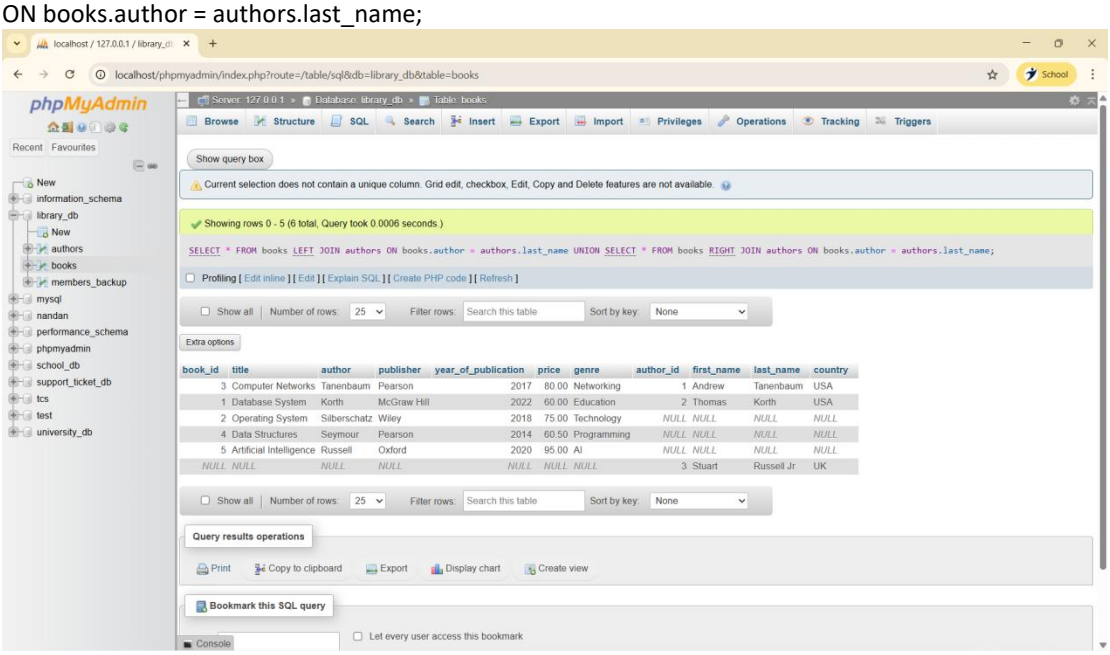
Lab 4: FULL OUTER JOIN

SELECT *
FROM books
LEFT JOIN authors
ON books.author = authors.last_name

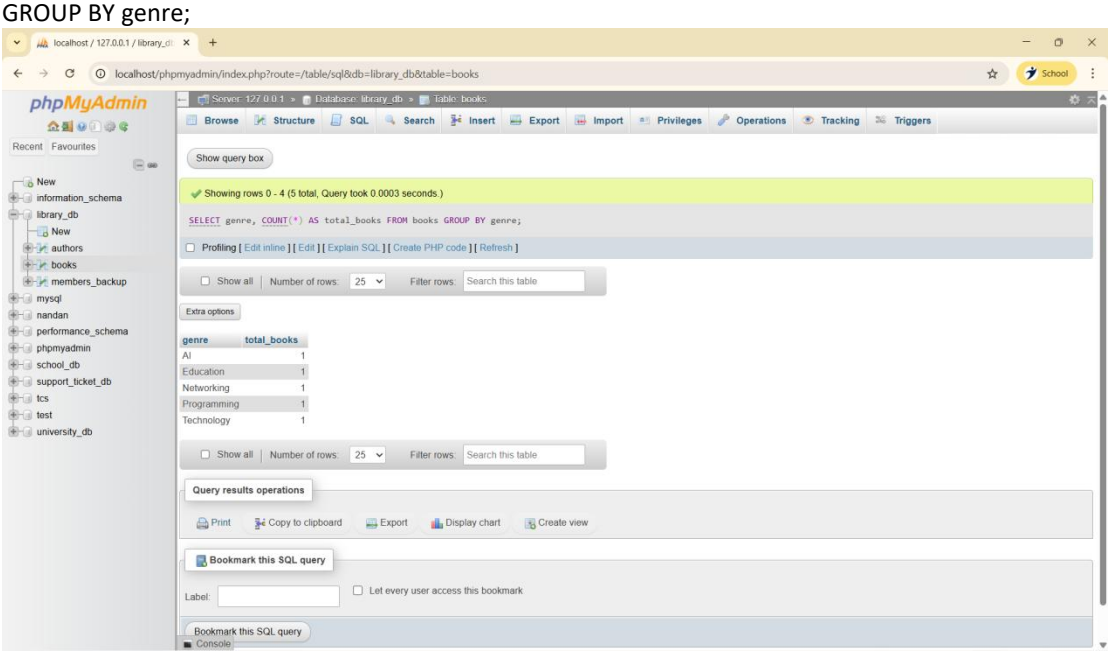
UNION

SELECT *
FROM books

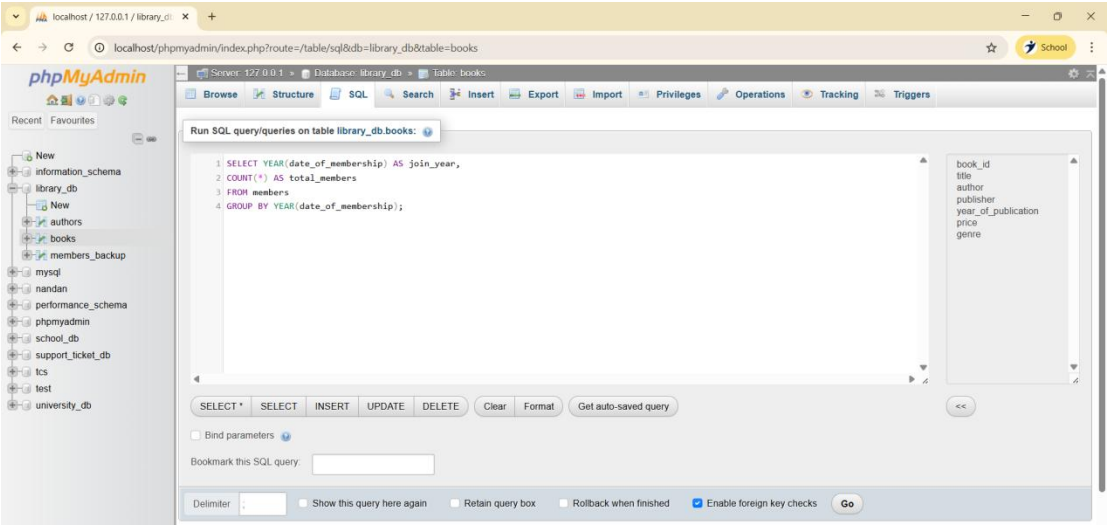
RIGHT JOIN authors
ON books.author = authors.last_name;



15. SQL GROUP BY
Lab 3: Count books by genre.
SELECT genre, COUNT(*) AS total_books
FROM books
GROUP BY genre;

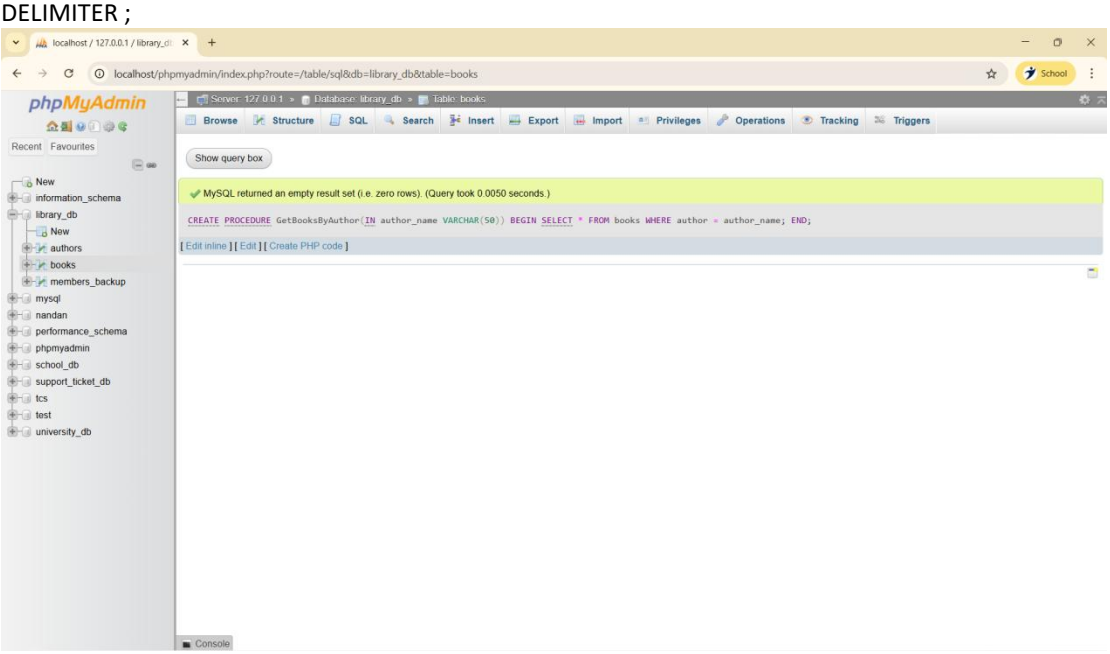


Lab 4: Members grouped by year joined.
SELECT YEAR(date_of_membership) AS join_year,
COUNT(*) AS total_members
FROM members
GROUP BY YEAR(date_of_membership);



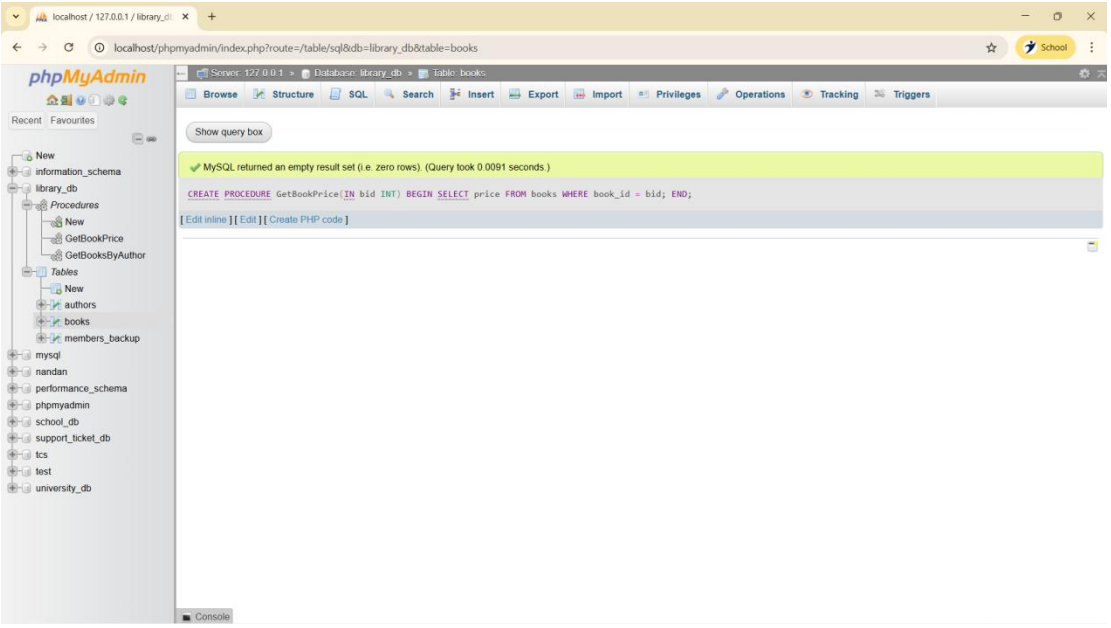
16. SQL Stored Procedure
Lab 3: Procedure to retrieve books by author.

```
DELIMITER $$

CREATE PROCEDURE GetBooksByAuthor(IN author_name VARCHAR(50))
BEGIN
SELECT * FROM books WHERE author = author_name;
END $$

DELIMITER ;
```

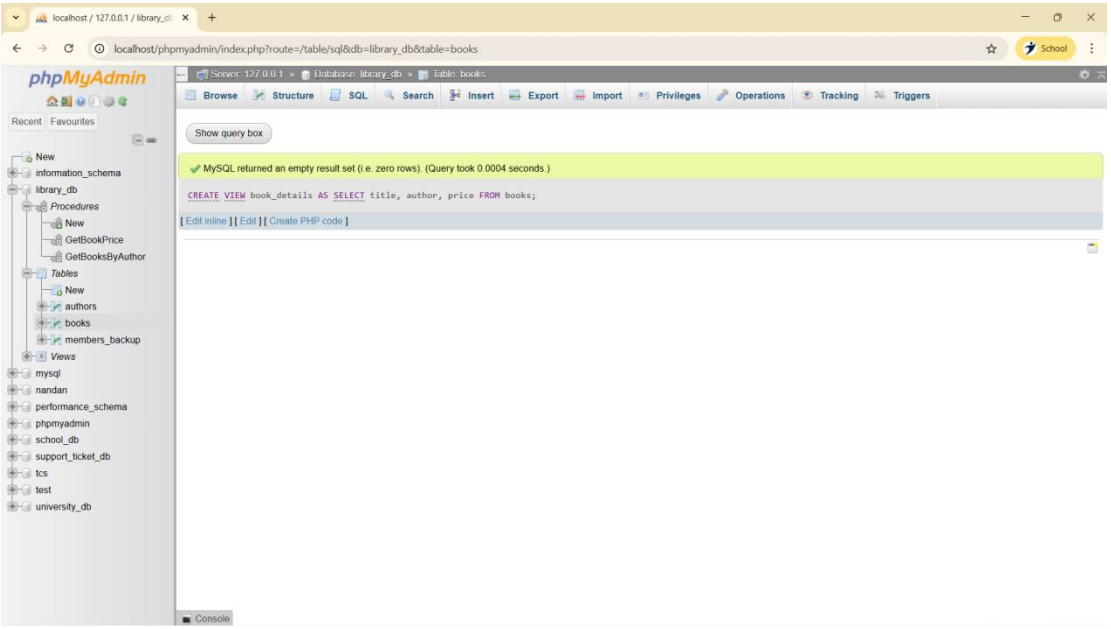


Lab 4: Procedure to return book price.

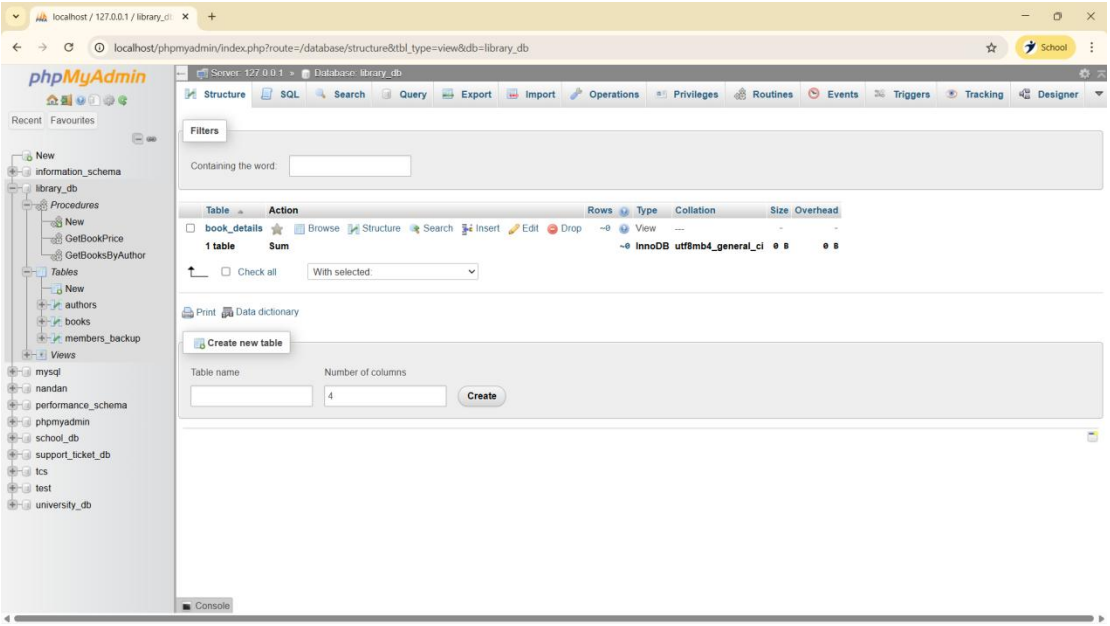
```
DELIMITER $$

CREATE PROCEDURE GetBookPrice(IN bid INT)
BEGIN
SELECT price FROM books WHERE book_id = bid;
END $$

DELIMITER ;
```



17. SQL View
Lab 3: View for title, author, price.
CREATE VIEW book_details AS
SELECT title, author, price
FROM books;



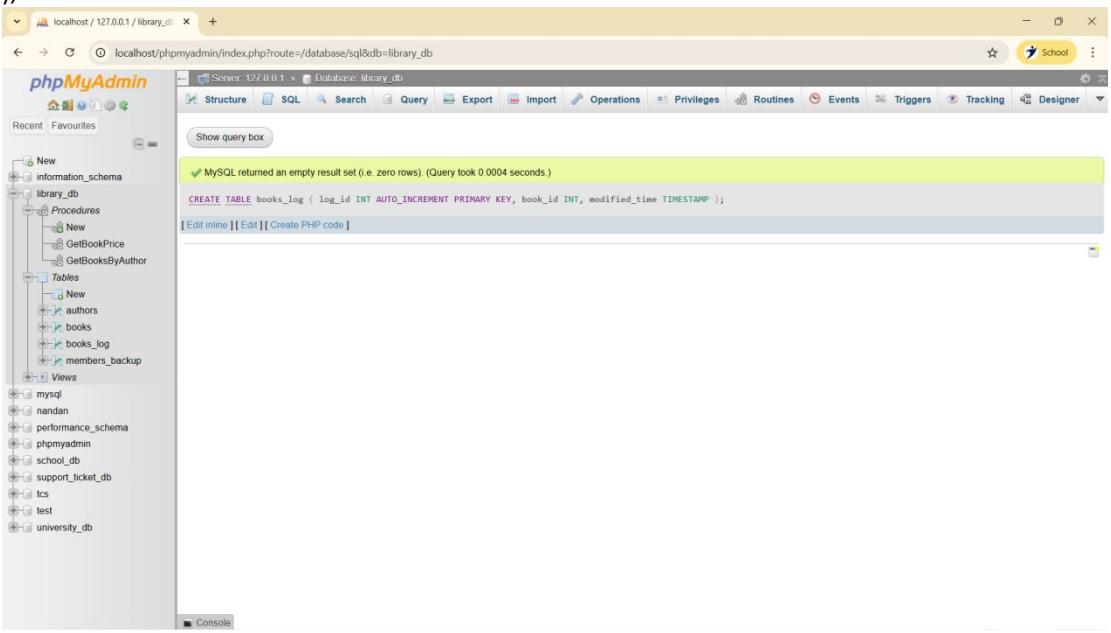
Lab 4: View for members before 2020.
CREATE VIEW old_members AS
SELECT * FROM members
WHERE date_of_membership < '2020-01-01';



18. SQL Trigger

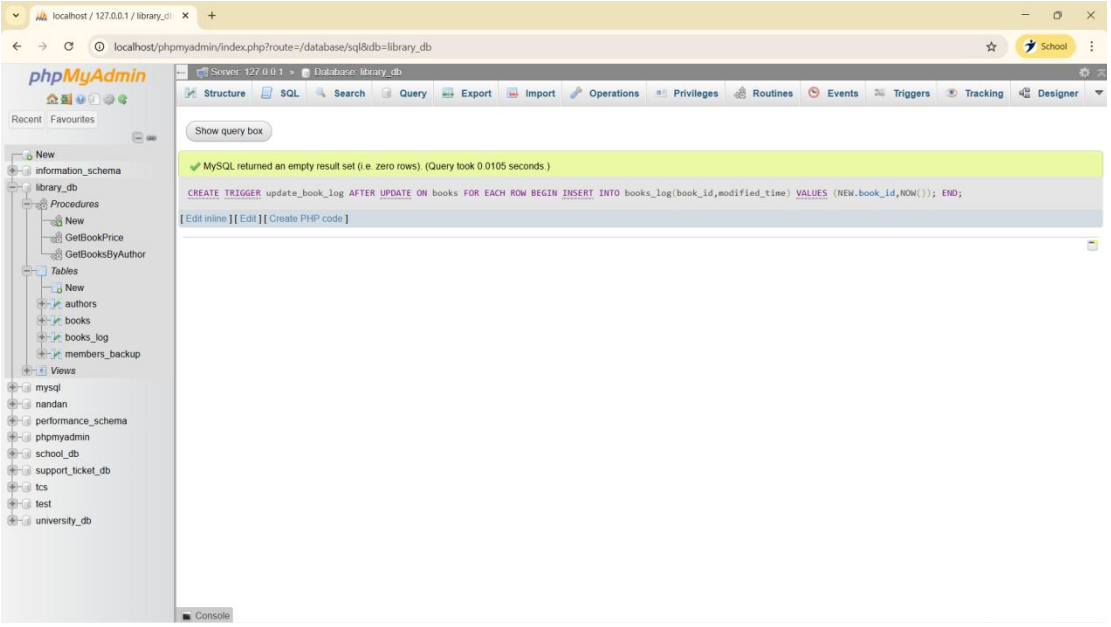
Lab 3: Trigger to update timestamp.

```
CREATE TABLE books_log (
log_id INT AUTO_INCREMENT PRIMARY KEY,
book_id INT,
modified_time TIMESTAMP
);
```



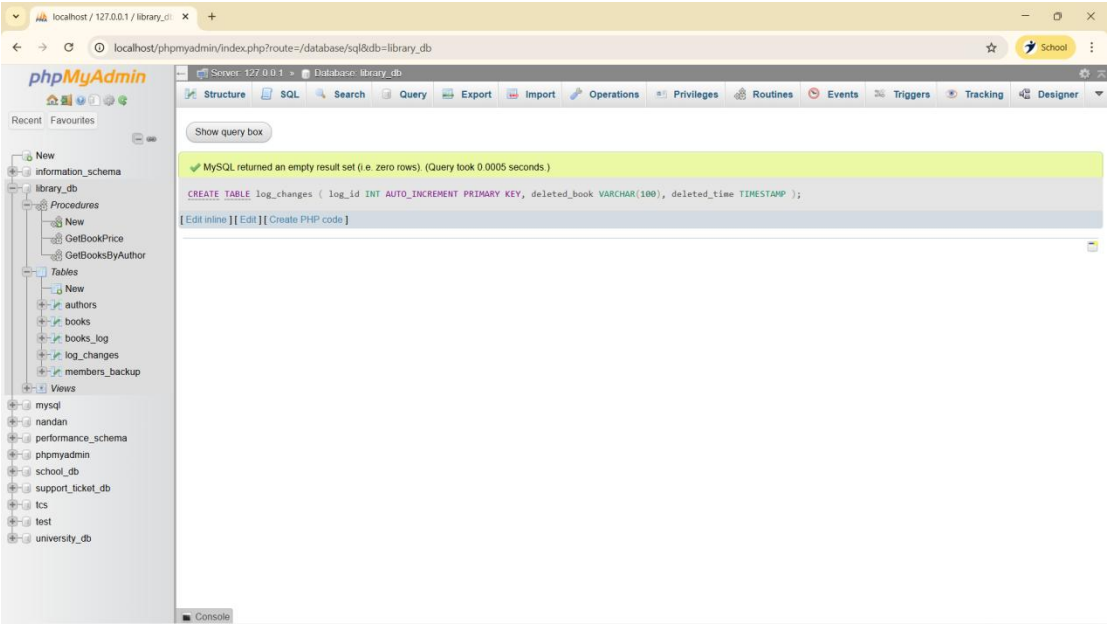

```
DELIMITER $$

CREATE TRIGGER update_book_log
AFTER UPDATE ON books
FOR EACH ROW
BEGIN
INSERT INTO books_log(book_id,modified_time)
VALUES (NEW.book_id,NOW());
END $$
```

DELIMITER ;



Lab 4: Trigger for DELETE log.

CREATE TABLE log_changes (
log_id INT AUTO_INCREMENT PRIMARY KEY,
deleted_book VARCHAR(100),
deleted_time TIMESTAMP
);

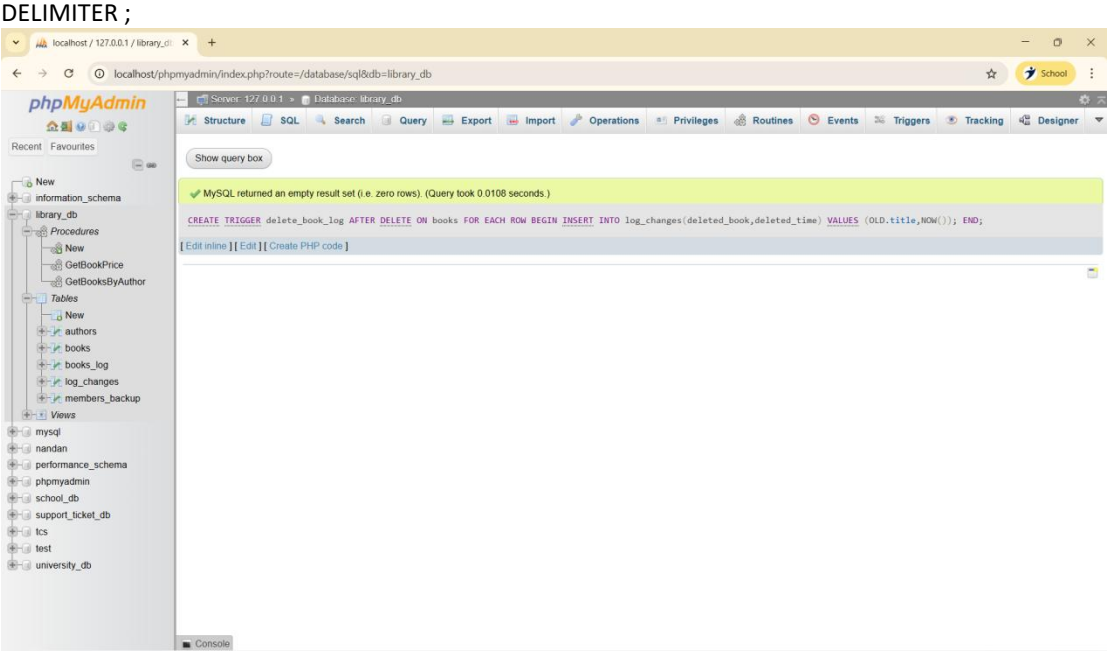


DELIMITER $$

CREATE TRIGGER delete_book_log
AFTER DELETE ON books
FOR EACH ROW
BEGIN
INSERT INTO log_changes(deleted_book,deleted_time)
VALUES (OLD.title,NOW());

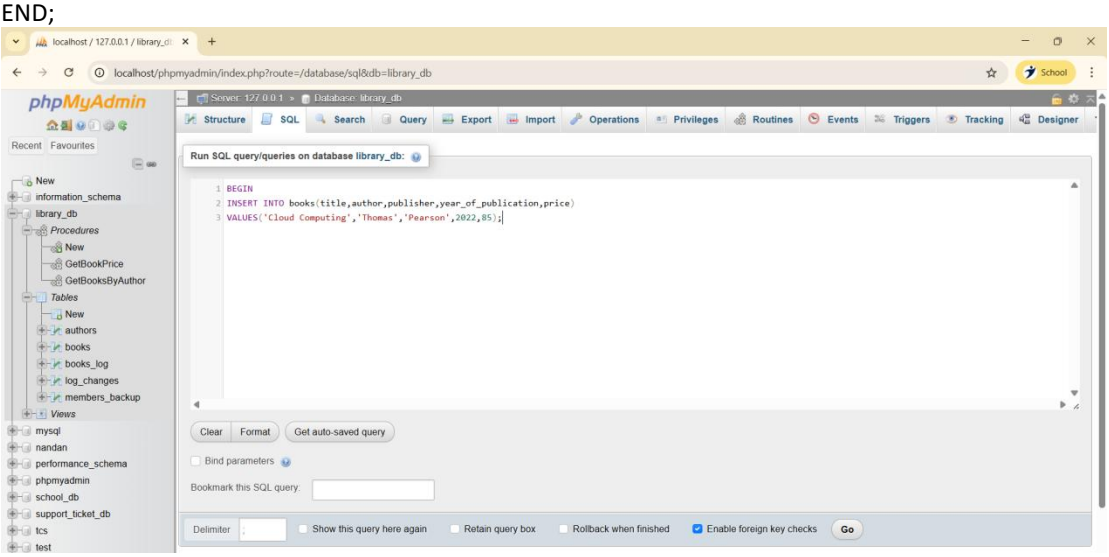
END $$

DELIMITER ;

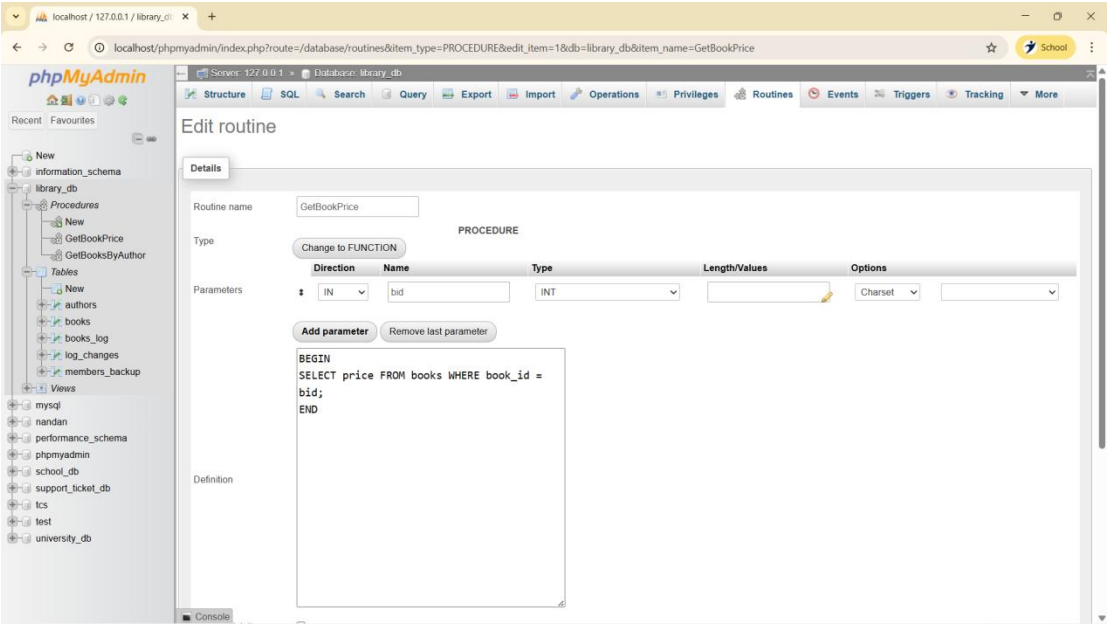


19. Introduction to PL/SQL
Lab 3: Insert book and display message.
BEGIN
INSERT INTO books(title,author,publisher,year_of_publication,price)
VALUES('Cloud Computing','Thomas','Pearson',2022,85);

DBMS_OUTPUT.PUT_LINE('Book inserted successfully');
END;



Lab 4: Display total books.
DECLARE
total_books NUMBER;
BEGIN
SELECT COUNT(*) INTO total_books FROM books;
DBMS_OUTPUT.PUT_LINE('Total Books: ' || total_books);
END;



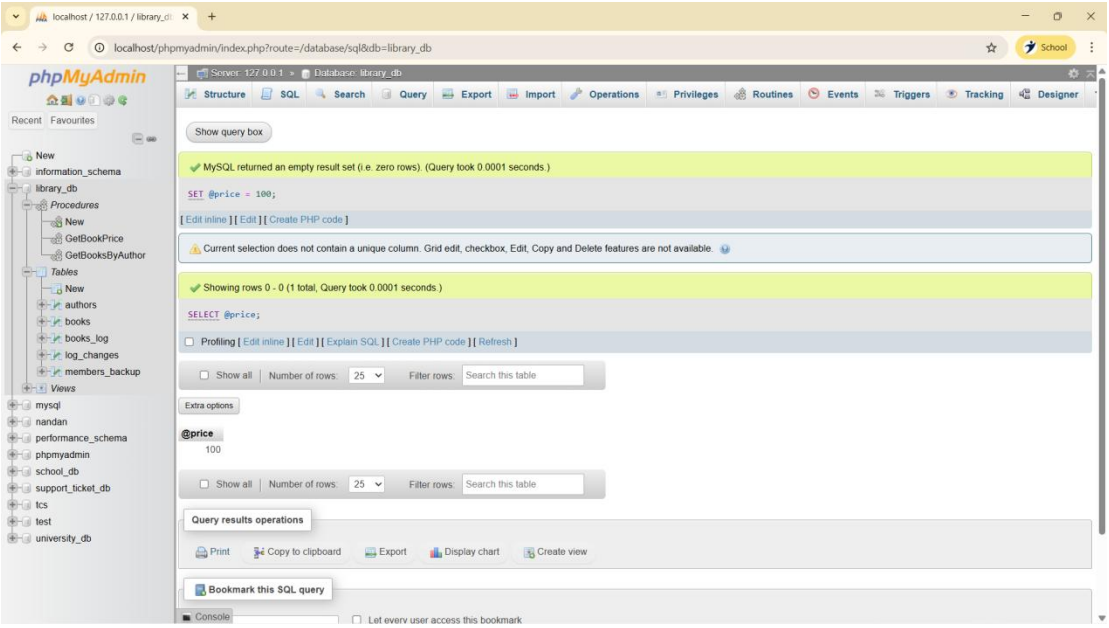
20. PL/SQL Syntax

Lab 3: Declare variables.

```
DECLARE
v_book_id NUMBER := 1;
v_price NUMBER := 75;
BEGIN
DBMS_OUTPUT.PUT_LINE('Book ID: ' || v_book_id);
DBMS_OUTPUT.PUT_LINE('Price: ' || v_price);
END;
```

Lab 4: Constants and arithmetic.

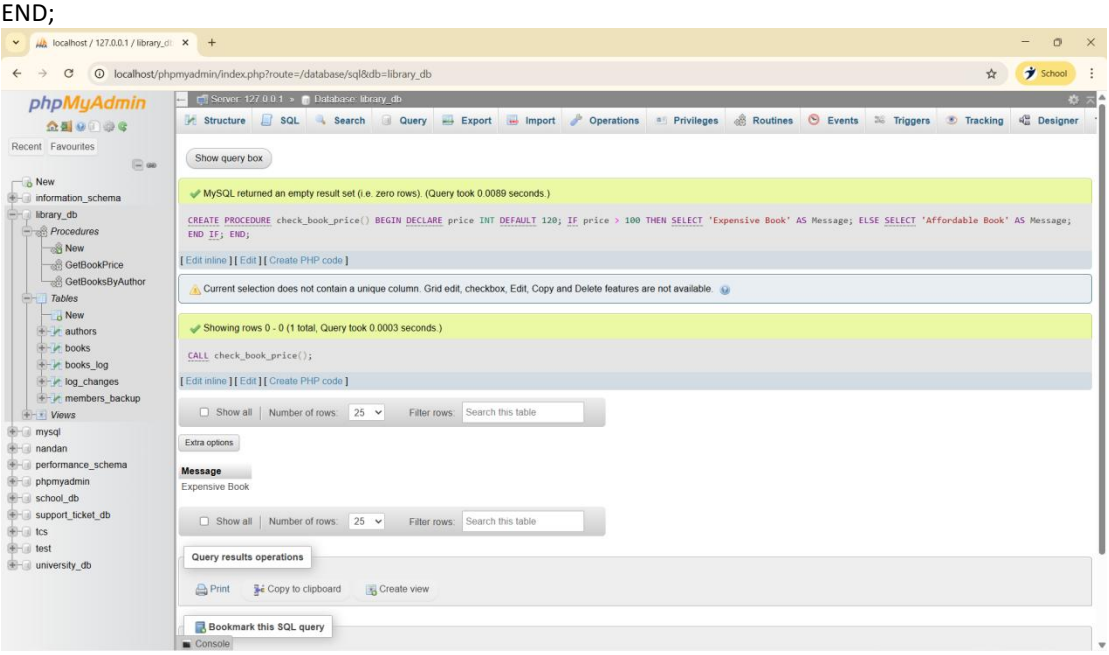
```
DECLARE
price NUMBER := 100;
discount CONSTANT NUMBER := 10;
final_price NUMBER;
BEGIN
final_price := price - discount;
DBMS_OUTPUT.PUT_LINE('Final Price: ' || final_price);
END;
```



21. PL/SQL Control Structures

Lab 3: IF condition.

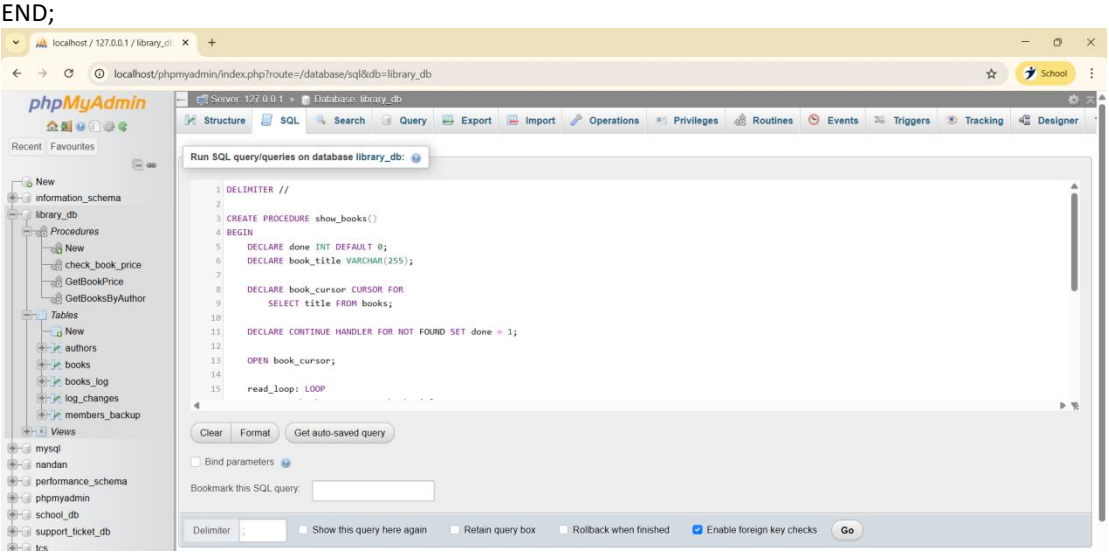
```
DECLARE
price NUMBER := 120;
BEGIN
IF price > 100 THEN
DBMS_OUTPUT.PUT_LINE('Expensive Book');
ELSE
DBMS_OUTPUT.PUT_LINE('Affordable Book');
END IF;
END;
```



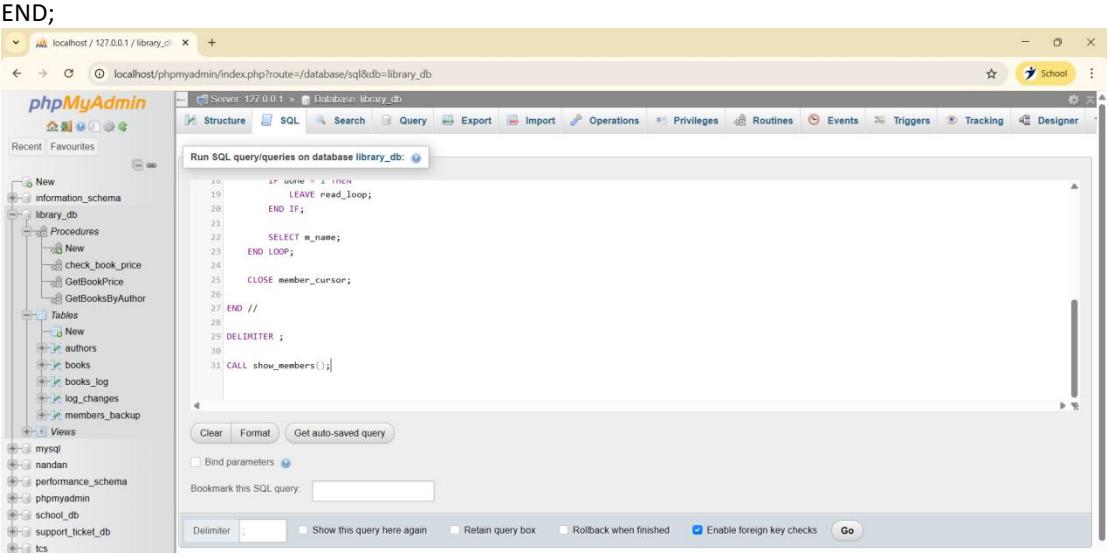
Lab 4: FOR LOOP example.

```
BEGIN
FOR rec IN (SELECT * FROM books)
LOOP
DBMS_OUTPUT.PUT_LINE(rec.title);
```

END LOOP;
END;



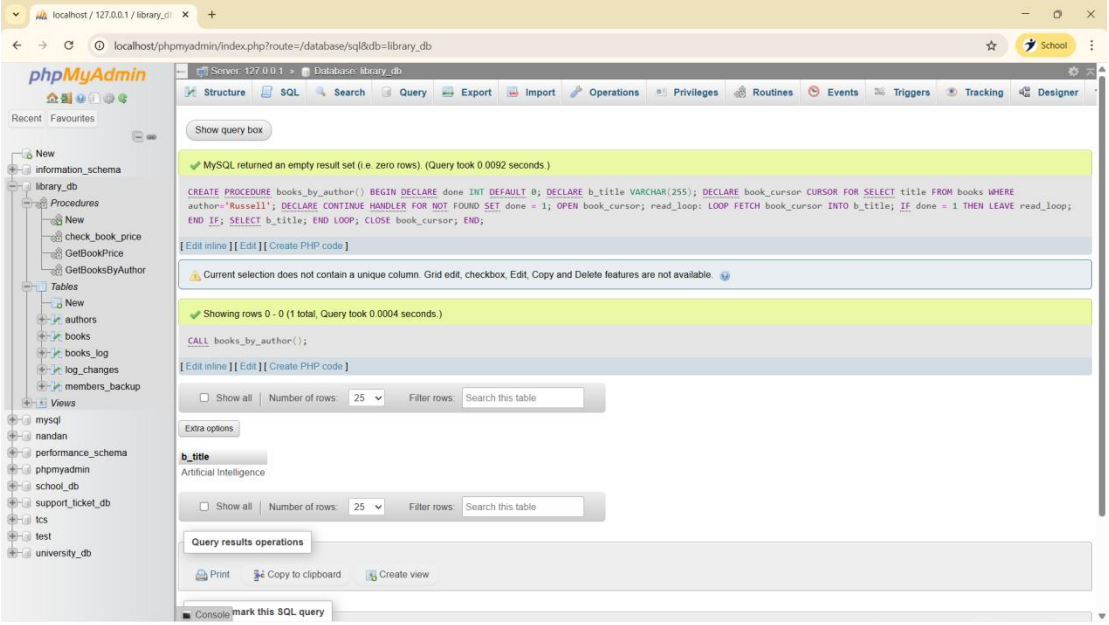
22. SQL Cursors
Lab 3: Explicit cursor example.
DECLARE
CURSOR member_cursor IS SELECT * FROM members;
m_rec members%ROWTYPE;
BEGIN
OPEN member_cursor;
LOOP
FETCH member_cursor INTO m_rec;
EXIT WHEN member_cursor%NOTFOUND;
DBMS_OUTPUT.PUT_LINE(m_rec.member_name);
END LOOP;
CLOSE member_cursor;
END;



Lab 4: Cursor for books by author.
DECLARE
CURSOR book_cursor IS
SELECT title FROM books WHERE author='Russell';

b_rec books%ROWTYPE;

```
BEGIN
OPEN book_cursor;
LOOP
FETCH book_cursor INTO b_rec;
EXIT WHEN book_cursor%NOTFOUND;
DBMS_OUTPUT.PUT_LINE(b_rec.title);
END LOOP;
CLOSE book_cursor;
END;
```



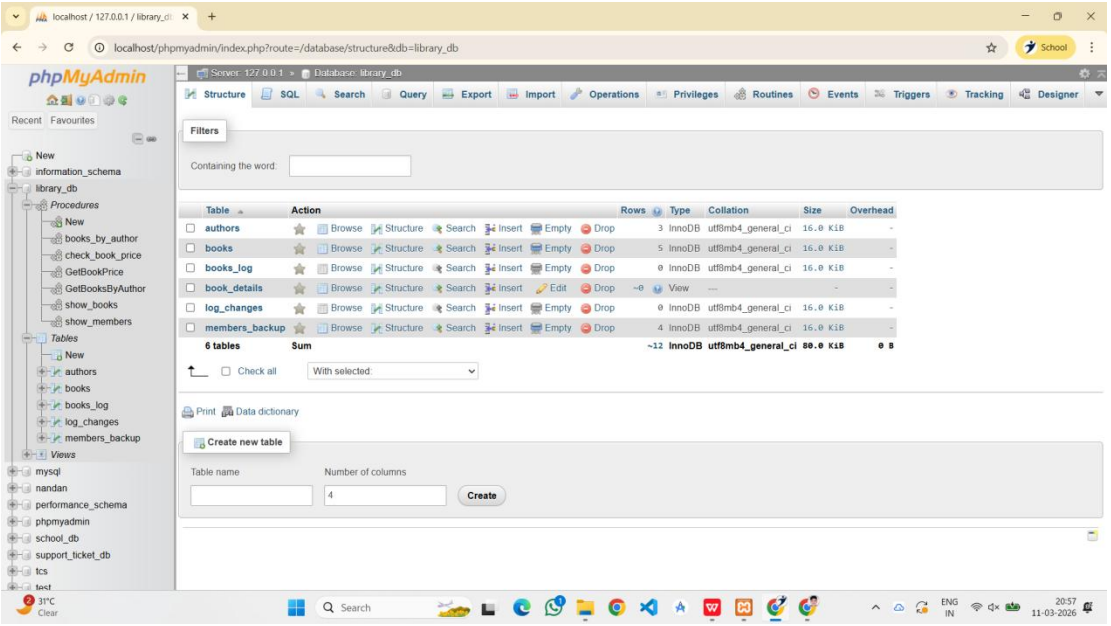
23. Rollback, Commit and Savepoint

Lab 3: Transaction with savepoint.

```
INSERT INTO members VALUES (NULL,'TEST USER','2024-01-01','test@gmail.com');

SAVEPOINT before_update;

UPDATE members SET member_name='Updated Test' WHERE member_id=1;

ROLLBACK TO before_update;
```

Lab 4: Commit and rollback example.

```
INSERT INTO books VALUES (NULL,'Cyber Security','Smith','Pearson',2023,90);
INSERT INTO books VALUES (NULL,'Blockchain','Nakamoto','Tech Press',2022,120);

COMMIT;

SAVEPOINT after_commit;

UPDATE books SET price=200 WHERE book_id=1;

ROLLBACK TO after_commit;
```


localhost / 127.0.0.1 / library_db
localhost/phpmyadmin/index.php?route=/database/structure&db=library_db
phpMyAdmin
Recent Favourites
New
information_schema
library_db
Procedures
New
books_by_author
check_book_price
GetBookPrice
GetBooksByAuthor
show_books
show_members
Tables
New
authors
books
books_log
log_changes
members_backup
Views
mysql
nandan
performance_schema
phpmyadmin
school_db
support_ticket_db
tcs
test
Server: 127.0.0.1 » Database: library_db
Structure SQL Search Query Export Import Operations Privileges Routines Events Triggers Tracking Designer
Filters
Containing the word:
Table Action Rows Type Collation Size Overhead
authors Browse Structure Search Insert Empty Drop 3 InnoDB utf8mb4_general_ci 16.0 KiB -
books Browse Structure Search Insert Empty Drop 5 InnoDB utf8mb4_general_ci 16.0 KiB -
books_log Browse Structure Search Insert Empty Drop 0 InnoDB utf8mb4_general_ci 16.0 KiB -
book_details Browse Structure Search Insert Edit Drop ~0 View ---- - -
log_changes Browse Structure Search Insert Empty Drop 0 InnoDB utf8mb4_general_ci 16.0 KiB -
members_backup Browse Structure Search Insert Empty Drop 4 InnoDB utf8mb4_general_ci 16.0 KiB -
6 tables Sum ~12 InnoDB utf8mb4_general_ci 80.0 KiB 0 B
Check all With selected:
Print Data dictionary
Create new table
Table name
Number of columns
4
Create
31°C Clear
Search
ENG IN
20:57 11-03-2026